



COMSATS University Islamabad, Pakistan

Estate Flow

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Muhammad Kaif

CIIT/FA22-BSE-031/SWL

Umar Raza

CIIT/FA22-BSE-064/SWL

Ali Hasnain

CIIT/FA22-BSE-067/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Kaif

Umar Raza

Ali Hasnain

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “EstateFlow” was developed by **Muhammad Kaif (CIIT/FA22-BSE-031/SWL)**, **Umar Raza (CIIT/FA22-BSE-064/SWL)** and **Ali Hasnain (CIIT/FA22-BSE-067/SWL)** under the supervision of Ms. Hafsa Saleem that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Ms. Hafsa Saleem

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Inam Illahi

Assistant Professor

Department of computer science and Emerging Technologies

Emerson University Multan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

EstateFlow is a mobile-based real estate co-investment platform developed to modernize property exploration, shared investment, and real estate management using interactive 3D visualization and cloud-based services. The application is designed to provide a centralized environment where buyers, investors, sellers, and 3D freelancers can collaborate efficiently. By combining mobile accessibility, real-time database integration, and immersive property viewing, EstateFlow simplifies property investment and improves decision-making for users.

The system allows users to register and log in securely, browse property listings, and explore properties through interactive 3D models. Buyers and investors can participate in co-investment opportunities, enabling multiple users to invest in a single property. This shared investment approach reduces financial barriers and promotes collaborative ownership. Users can also manage their investments, track profit and loss, and view analytics that provide insights into their financial performance. These features enhance transparency and support informed investment decisions.

For sellers and brokers, EstateFlow provides tools to upload and manage property listings, add images, and associate 3D models with properties. Sellers can monitor investment participation and generate ledger reports for financial tracking. The system also supports 3D freelancers who can upload property models to enhance visualization. These models are rendered using Flutter-compatible 3D visualization libraries, allowing users to rotate, zoom, and explore properties interactively within the mobile application.

The EstateFlow application is developed using Flutter framework with Dart programming language, ensuring cross-device compatibility and responsive user interface. Firebase backend services are used for authentication, cloud database management, and file storage. Firebase Authentication ensures secure user access, Cloud Firestore stores structured data including users and investments, and Firebase Cloud Storage manages property images and 3D models. This architecture provides scalability, real-time updates, and reliable data handling.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor of Ms. Hafsa Saleem. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply thankful for their encouragement and continual support during this work.

We also extend our heartfelt gratitude to our families and friends who have been a constant source of support and motivation throughout this journey.

Muhammad Kaif

Umar Raza

Ali Hasnain

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
MVC	Model-View-Controller
NoSQL	Non-Structured Query Language
REST	Representational State Transfer
UI	User Interface

Table of Contents

1 Introduction.....	1
1.1 Brief.....	1
1.1.1 Purpose of the Project.....	2
1.1.2 Scope of the Project.....	3
1.1.3 Technologies Used.....	3
1.1.4 System Overview.....	4
1.1.5 Outcome of the Project.....	5
1.1 Relevance to Course Modules.....	5
1.2 Project Background.....	6
1.3 Literature Review.....	7
1.4 Analysis from Literature Review.....	8
1.5 Methodology and Software Lifecycle.....	8
1.5.1 Rationale behind Selected Methodology.....	9
2 Problem Definition.....	12
2.1 Problem Statement.....	12
2.2 Deliverables and Development Requirements.....	13
2.3 Current System.....	14
3 Requirement Analysis.....	17
3.1 Use Case Diagram.....	17
3.2 Use Case Description.....	18
3.3 Functional Requirements.....	27
3.4 Non-Functional Requirements.....	28
4 Design and Architecture.....	34
4.1 System Architecture.....	34
4.2 Data Representation.....	35
4.2.1 Key Entities and Relationships.....	36
4.3 Process Flow/Representation.....	38
4.3.1 Activity Diagrams.....	39
4.4 Design Models.....	43
4.4.1 Sequence Diagram.....	43
4.4.2 Class Diagram.....	48
5 Implementation.....	51
5.1 Algorithm.....	51
5.1.1 User authentication.....	51
5.1.2 Upload Property Listing.....	52
5.1.3 Co-Invest in Property.....	53
5.1.4 Upload 3D Property Model.....	54

5.2 External APIs.....	56
5.3 User Interface	57
5.3.2 Login Screen.....	59
5.3.3 Dashboard.....	60
5.3.4 Profile Manage Screen.....	61
5.3.5 Add Sale Property Screen.....	62
5.3.6 Add Purchase Property Screen	63
5.3.7 Available Property Screen	64
6 Testing and Evaluation.....	66
6.1 Manual Testing.....	66
6.1.1 System Testing	67
6.1.2 Unit Testing.....	68
6.1.3 Functional Testing.....	69
6.1.4 Integration Testing.....	70
6.2 Tools and Technologies.....	71
6.3 Summary.....	72
7 Conclusion	74
7.1 Future Work.....	75
8 References.....	78

List of Figures

FIGURE 3. 1: ESTATE FLOW USE CASE.....	17
FIGURE 4. 1: ESTATEFLOW SYSTEM ARCHITECTURE.....	35
FIGURE 4. 2: ESTATEFLOW ER DIAGRAM.....	37
FIGURE 4. 3: USER LOGIN AND REGISTRATION ACTIVITY DIAGRAM	39
FIGURE 4. 4: UPLOAD PROPERTY LISTING (SELLER/BROKER) ACTIVITY DIAGRAM.....	40
FIGURE 4. 5: CO-INVEST IN PROPERTY (BUYER/INVESTOR) ACTIVITY DIAGRAM	41
FIGURE 4. 6: UPLOAD & EXPLORE 3D PROPERTY MODEL ACTIVITY DIAGRAM	42
FIGURE 4. 7: SEQUENCE DIAGRAM FOR USER REGISTRATION AND LOGIN	44
FIGURE 4. 8: UPLOAD PROPERTY LISTING (SELLER/BROKER) SEQUENCE DIAGRAM	45
FIGURE 4. 9: CO-INVEST IN PROPERTY (BUYER/INVESTOR) SEQUENCE DIAGRAM.....	46
FIGURE 4. 10: UPLOAD & EXPLORE 3D PROPERTY MODEL SEQUENCE DIAGRAM	47
FIGURE 4. 11: CLASS DIAGRAM	48
FIGURE 5. 1: SIGN-UP SCREEN	58
FIGURE 5. 2: LOGIN SCREEN	59
FIGURE 5. 3:DASHBOARD SCREEN	60
FIGURE 5. 4: PROFILE SCREEN	61
FIGURE 5. 5: ADD SALE PROPERTY SCREEN.....	62
FIGURE 5. 6: ADD PURCHASE PROPERTY SCREEN	63
FIGURE 5. 7: AVAILABLE PROPERTY SCREEN.....	64

LIST OF TABLES

TABLE 1. 1: RELEVANCE TO COURSE MODULES	5
TABLE 3. 1: EXPLORE PROPERTY IN 3D VIEW	18
TABLE 3. 2: MANAGE INVESTMENTS.....	19
TABLE 3. 3: CO-INVEST IN PROPERTY	20
TABLE 3. 4: UPLOAD / MANAGE PROPERTY LISTINGS	21
TABLE 3. 5: UPLOAD 3D PROPERTY MODEL	22
TABLE 3. 6: REGISTER / LOGIN.....	23
TABLE 3. 7: VIEW INVESTMENT ANALYTICS.....	24
TABLE 3. 8: TRACK PROFIT / LOSS	25
TABLE 3. 9: GENERATE & PRINT LEDGER	26
TABLE 5. 1: EXTERNAL APIS	57
TABLE 6. 1: SYSTEM TESTING TABLE	67
TABLE 6. 2: UNIT FUNCTION TABLE	68
TABLE 6. 3: TEST CASE TABLE	69
TABLE 6. 4: INTEGRATION TESTING TABLE.....	70
TABLE 6. 5: TOOLS AND TECHNOLOGIES TABLE	71

Chapter 1

INTRODUCTION

1 Introduction

The rapid advancement of digital technologies has transformed the way real estate transactions and investments are conducted. Modern buyers and investors expect interactive platforms that provide transparency, analytics, and collaborative investment opportunities. Traditional real estate systems mainly rely on static listings, manual coordination, and limited financial insights, which often lead to inefficient decision-making and reduced accessibility for small investors.

EstateFlow addresses these limitations by introducing an integrated real-estate co-investment and property visualization platform. The system enables users to explore properties interactively, participate in shared investments, track financial performance, and manage real estate portfolios within a single environment. EstateFlow also supports 3D property visualization, allowing buyers and investors to virtually explore properties before making investment decisions. Additionally, sellers and brokers can upload property listings, while 3D freelancers can contribute immersive models to enhance property presentation.

Developed using modern technologies and modular architecture, EstateFlow provides scalability, secure authentication, and efficient data management. The platform includes modules for property management, co-investment handling, analytics generation, profit/loss tracking, and ledger reporting. These components work together to provide a transparent and collaborative investment ecosystem.

In essence, EstateFlow demonstrates how real estate investment, financial tracking, and 3D visualization can be combined into a unified system that improves accessibility, transparency, and decision-making for buyers, investors, and real estate professionals.

1.1 Brief

EstateFlow is a mobile-based real estate co-investment platform designed to transform traditional property investment into a collaborative and interactive digital experience. The application enables users to browse properties, explore them using immersive 3D visualization, and invest in properties through shared ownership. The main goal of the system is to reduce financial barriers in real estate investment by allowing multiple investors to contribute to a single property. This approach not only increases accessibility but also provides users with flexible investment opportunities.

The application supports multiple user roles including Buyer/Investor, Seller/Broker, and 3D Freelancer. Buyers and investors can view property listings, analyze investment opportunities, and co-invest in properties. Sellers and brokers can upload and manage property listings, monitor investment participation, and generate ledger reports. 3D freelancers are responsible for uploading 3D property models that enhance visualization and provide a realistic property viewing experience. These models allow users to rotate, zoom, and navigate through properties interactively before making investment decisions.

EstateFlow is developed as a mobile application using Flutter framework and Dart programming language, providing a responsive and user-friendly interface. Firebase services are used as backend infrastructure, including Firebase Authentication for secure login, Cloud Firestore for database management, and Firebase Cloud Storage for storing property images and 3D models. The application also includes modules for investment tracking, profit and loss calculation, analytics generation, and ledger management. By combining mobile accessibility, cloud-based services, and interactive visualization, EstateFlow provides a modern solution for collaborative real estate investment

1.1.1 Purpose of the Project

The purpose of the EstateFlow project is to develop a mobile application that simplifies and modernizes the process of real estate investment. Traditional real estate systems often require users to purchase entire properties, which can be financially challenging and time-consuming. Additionally, most platforms provide static property images that do not offer a complete understanding of the property. EstateFlow addresses these limitations by introducing a collaborative investment model and interactive 3D visualization.

The system aims to allow multiple investors to co-invest in a single property, enabling shared ownership and reducing financial barriers. This approach provides flexibility for users who want to invest smaller amounts while still participating in real estate opportunities. Another purpose of the project is to enhance decision-making through 3D property visualization. Users can explore properties virtually and understand layouts before investing.

The project also aims to provide transparency in financial management. Users can track their investments, calculate profit and loss, and view analytics. Sellers can monitor investor participation and generate ledger reports. Additionally, the system aims to create a role-based environment where buyers, investors, sellers, and freelancers collaborate efficiently.

Overall, the purpose of EstateFlow is to provide a secure, scalable, and user-friendly mobile platform for collaborative property investment.

1.1.2 Scope of the Project

The scope of the EstateFlow project includes the development of a mobile application that supports property browsing, co-investment, 3D visualization, and investment tracking. The system allows users to register and log in securely using Firebase Authentication. Buyers and investors can browse available properties and explore them in 3D view. Users can invest in properties by entering an investment amount, and the system calculates ownership share automatically.

The system also provides property management functionality for sellers and brokers. Sellers can upload property details including title, location, price, description, and images. They can also monitor investment participation and generate financial ledger reports. The platform supports 3D freelancers who upload property models, which are stored in Firebase Cloud Storage and linked to properties.

EstateFlow also includes analytics and profit/loss tracking modules. Users can view their investment portfolio, analyze returns, and monitor performance. The system ensures real-time updates using Cloud Firestore database. The application is designed primarily for Android mobile devices and focuses on real estate investment and visualization.

The scope does not include legal property ownership transfer, financial institution integration, or government verification processes. Payment processing and contract management are also outside the current scope. However, these features may be added in future versions.

1.1.3 Technologies Used

EstateFlow is developed using modern mobile and cloud-based technologies to ensure performance, scalability, and reliability. The frontend of the application is built using Flutter framework, which allows development of responsive and interactive mobile user interfaces. Dart programming language is used to implement application logic and manage UI components.

Firebase Authentication is used to handle secure user registration and login. It provides role-based access control and ensures that only authorized users can access system features. Cloud Firestore is used as the primary database to store structured data including users,

properties, investments, analytics, and ledger records. Firebase Cloud Storage is used for storing property images and 3D models uploaded by sellers and freelancers.

Flutter-compatible 3D visualization libraries are integrated to render interactive property models. These libraries allow users to explore properties using gestures such as zoom, rotate, and pan. Visual Studio Code and Android Studio are used as development tools. GitHub is used for version control and collaboration. Firebase SDK is used to connect Flutter application with backend services.

The system also follows Android mobile application guidelines for deployment. These technologies collectively provide a scalable architecture, real-time data synchronization, and secure user experience.

1.1.4 System Overview

EstateFlow is a role-based mobile application designed to support collaborative property investment and interactive visualization. The system consists of three main user roles: Buyer/Investor, Seller/Broker, and 3D Freelancer. Each role interacts with the system through dedicated dashboards and functionalities.

Buyers and investors can browse property listings, view details, and explore properties in 3D view. They can also co-invest in properties by entering investment amounts. The system calculates ownership share and stores investment data. Users can manage their investment portfolio, view analytics, and track profit or loss.

Sellers and brokers are responsible for uploading property listings. They enter property details and upload images. Sellers can also monitor investment participation and generate ledger reports. This helps maintain transparency and financial tracking.

3D freelancers upload 3D property models, which are stored in Firebase Cloud Storage. These models are linked to properties and rendered in the mobile application. The system retrieves model data from Firestore and loads them using 3D viewer.

The application uses Flutter UI layer for interaction, Firebase Authentication for login, Cloud Firestore for database, and Firebase Storage for files. The architecture ensures modular design and real-time updates.

1.1.5 Outcome of the Project

The outcome of the EstateFlow project is a fully functional mobile application that enables collaborative real estate investment and interactive property exploration. The system successfully integrates Flutter mobile frontend with Firebase backend services. Users can register, browse properties, explore them in 3D, and invest collaboratively.

The application provides investment tracking, analytics, and profit/loss calculation features. Sellers can upload and manage property listings. Freelancers can upload 3D models. The system ensures secure authentication and real-time database synchronization.

The project demonstrates the use of modern mobile technologies to solve real estate investment challenges. The 3D visualization feature enhances user understanding of properties. The co-investment functionality reduces financial barriers. The analytics module provides insights into investment performance.

Overall, the EstateFlow project delivers a scalable, user-friendly, and interactive mobile application. The system improves accessibility, transparency, and efficiency in real estate investment. It serves as a complete digital solution for collaborative property investment and management.

1.1 Relevance to Course Modules

The development of the EstateFlow platform integrates multiple concepts, theories, and technical skills acquired throughout the Bachelor of Science in Computer Science (BSCS) degree program. This project combines software engineering, database management, system design, web development, and human-computer interaction to build a comprehensive real estate investment and property visualization platform. Each course module contributes to different aspects of the system as outlined below.

Table 1. 1: Relevance to Course Modules

Course Module	Application in EstateFlow
Software Engineering	Preparation of SRS & SDD documents, requirement analysis, UML diagrams

Database Systems	Designed databases for users, properties, investments, and ledger records
Object Oriented Programming	Implemented system classes such as User, Property, Investment, and Analytics
Mobile Application Development	Developed interactive UI for buyers, investors, sellers, and freelancers
Human Computer Interaction (HCI)	Designed user-friendly dashboards and 3D property navigation
Software Design & Architecture	Created layered architecture and service-based modules
Data Structures	Used for managing property lists, investment records, and analytics data
Software Testing	Performed validation of login, property upload, and co-investment processes
Information Security	Implemented authentication and role-based access control

Table 1.1.1 outlines how the project reinforces theoretical understanding while demonstrating practical implementation of interdisciplinary knowledge aligned with Computer Science curriculum objectives. By integrating these subjects, the project provides a comprehensive demonstration of real-world application of theoretical learning.

1.2 Project Background

The real estate industry traditionally requires large capital investment, which limits participation to a small group of investors. Many individuals interested in real estate investment are unable to participate due to financial constraints. Furthermore, conventional property listing platforms only provide images and descriptions, which often fail to represent the actual property structure and value.

EstateFlow was developed to overcome these limitations by introducing co-investment functionality and 3D property exploration. The system allows multiple investors to collectively invest in a single property, reducing financial barriers and enabling shared ownership. Each investor receives a proportional share based on their contribution, and the system automatically tracks investments and calculates profit or loss.

The platform also integrates 3D property visualization, enabling buyers and investors to virtually explore properties before investing. Sellers and brokers can upload property details, while 3D freelancers can provide immersive models to improve property presentation. Additionally, analytics dashboards provide insights into investment performance, while ledger generation ensures financial transparency.

This project demonstrates how a digital platform can modernize real estate investment by combining property management, co-investment, financial tracking, and interactive visualization into a unified system.

1.3 Literature Review

The adoption of digital platforms in the real estate industry has significantly improved property management and investment accessibility. Existing platforms such as Zillow, Realtor.com, and Property Finder provide property listings, filtering options, and location-based searches. However, these systems mainly focus on property discovery and lack collaborative investment and financial tracking features.

Research studies on fractional real estate investment highlight that shared ownership models increase accessibility for small investors. These platforms allow multiple users to invest in a property and receive returns proportional to their investment. Such systems improve capital distribution and reduce investment risk.

Another area of research focuses on 3D visualization in real estate platforms. Virtual walkthroughs and interactive models allow users to better understand property layout and structure. Studies show that 3D visualization improves decision-making and enhances user confidence.

Additionally, financial analytics in investment platforms provide transparency by tracking returns, profit/loss, and portfolio performance. However, most existing systems implement only limited analytics features and lack integrated ledger generation.

Although existing solutions provide property listing, visualization, or investment tracking individually, very few platforms integrate all these features into a single system. Therefore, there is a need for a unified platform that supports co-investment, 3D visualization, analytics, and financial tracking.

1.4 Analysis from Literature Review

From the literature reviewed, it is evident that digital transformation in real estate improves accessibility and decision-making. However, existing systems lack integration of multiple investment and visualization features. The following key insights were derived:

- Traditional real estate systems focus only on property listings
- Fractional investment functionality is limited or absent
- 3D property visualization improves user decision-making
- Financial tracking increases transparency and trust
- Integrated analytics dashboards improve investment monitoring
- Unified systems provide better user experience and efficiency

Based on this analysis, EstateFlow was conceptualized as a complete real estate ecosystem combining property listing, co-investment, analytics, profit tracking, and 3D visualization. The system aims to improve transparency, accessibility, and decision-making for all stakeholders.

1.5 Methodology and Software Lifecycle

The Agile Software Development Lifecycle (SDLC) was adopted for the EstateFlow project to ensure flexibility, iterative development, and continuous feedback. Agile methodology supports incremental development and allows modifications based on evolving system requirements.

Key Phases:

1. Requirement Gathering

Identification of system requirements including user roles, property listing, co-investment, 3D visualization, analytics, and ledger generation.

2. System Design

Creation of UML diagrams, system architecture, database schema, and interface prototypes for EstateFlow modules.

3. Implementation

Development of system modules including authentication, property management, investment handling, analytics, and 3D model integration.

4. Module Integration

Integration of property service, investment service, analytics service, and ledger generation module into a unified platform.

5. Testing and Validation

Execution of unit testing, integration testing, and system testing to verify correctness and performance.

6. Deployment

Deployment of the EstateFlow platform to enable user access and real-time property interaction.

7. Maintenance

Continuous updates, bug fixing, and enhancement of analytics and visualization features.

1.5.1 Rationale behind Selected Methodology

The Agile methodology was selected due to its iterative and incremental nature, allowing continuous improvement throughout the EstateFlow development lifecycle. Unlike traditional waterfall models, Agile supports flexible requirement changes, regular stakeholder feedback, and parallel development of independent modules.

EstateFlow consists of multiple components including property management, co-investment, analytics, and 3D visualization. Agile enables these modules to be developed independently and integrated gradually. This approach reduces risk, improves quality, and ensures the system meets user requirements.

Therefore, Agile methodology is well suited for developing EstateFlow as a scalable, modular, and user-centered real estate platform.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

The real estate industry is rapidly evolving with the adoption of mobile applications that provide instant access to property information and investment opportunities. However, most existing real estate mobile applications focus only on property listings and lack advanced features such as collaborative investment, financial tracking, and interactive property visualization. This limitation makes it difficult for users to evaluate investment opportunities and participate in shared ownership.

Traditional real estate apps also rely on static images and textual descriptions, which do not provide a realistic view of properties. Investors and buyers often need better visualization tools to understand property layout and structure. Additionally, these systems do not provide investment analytics, profit/loss tracking, or transparent financial records.

The EstateFlow project was initiated to address these limitations by introducing a mobile-based real estate co-investment platform developed using Flutter and Dart. The application integrates Firebase backend services for authentication, cloud database, and storage. The system allows users to explore properties in 3D, co-invest with other investors, track financial performance, and manage real estate portfolios directly from a mobile application.

EstateFlow connects buyers, investors, sellers/brokers, and 3D freelancers into a unified mobile ecosystem that supports shared ownership, interactive visualization, and transparent financial management.

2.1 Problem Statement

Existing real estate mobile applications mainly provide basic property listings and manual search features. These systems lack co-investment functionality, analytics dashboards, and financial tracking mechanisms. As a result, users face difficulties in evaluating property value, calculating investment share, and tracking profit or loss.

Furthermore, most applications do not provide immersive property viewing. Users must rely on images, which reduces decision-making confidence. Sellers and brokers also lack tools to manage investor participation and maintain transparent records.

Key problems identified include:

- Lack of co-investment functionality for shared property ownership

- Absence of 3D property visualization for interactive exploration
- No investment analytics for tracking returns and performance
- Limited transparency in profit and loss calculation
- No automated ledger generation for financial records
- Manual property and investment management processes
- Poor collaboration between buyers, investors, and sellers
- Lack of centralized platform for real estate investment tracking

Hence, there is a clear need for an integrated real estate system that supports collaborative investment, interactive property visualization, financial analytics, and transparent management.

2.2 Deliverables and Development Requirements

The EstateFlow project aims to deliver a complete, fully functional real estate co-investment and property visualization platform. The system integrates property listing management, shared investment handling, analytics dashboards, and financial tracking into a unified architecture.

Major Deliverables:

- **EstateFlow Mobile Application** – allowing users to browse properties, co-invest, and explore 3D models
- **Property Management Module** – enabling sellers and brokers to upload and manage listings
- **Co-Investment Module** – supporting shared ownership and investment tracking
- **3D Property Visualization Module** – allowing immersive property exploration
- **Investment Analytics Dashboard** – providing insights into returns and performance
- **Profit/Loss Tracking System** – calculating investor gains and losses
- **Ledger Generation Module** – producing financial records and reports
- **Secure Authentication System** – role-based login for users
- **Technical Documentation and Source Code Repository** – for maintenance and

- scalability

Development Requirements:

- o Mobile Framework: Flutter
- o Programming Language: Dart
- o Backend Services: Firebase
- o Authentication: Firebase Authentication
- o Database: Cloud Firestore
- o Storage: Firebase Cloud Storage
- o 3D Visualization: Flutter-compatible 3D viewer / external API
- o Architecture: Mobile layered architecture
- o Deployment Platform: Android (Google Play Store)
- o Tools: Android Studio, Visual Studio Code, Firebase Console
- o Version Control: Git

The application must use Firebase for backend services, including authentication, cloud database (Firestore), and cloud storage. The app's 3D property viewing feature integrates with Flutter-compatible 3D visualization libraries or external APIs. The system also complies with Google Play Store publishing policies and Android mobile security guidelines.

Together, these deliverables ensure that EstateFlow functions as a scalable, secure, and interactive mobile real estate investment platform.

2.3 Current System

In the current scenario, most real estate mobile applications provide limited functionality such as property listings, search filters, and static property descriptions. These applications do not support co-investment or shared ownership, which restricts users from participating in high-value property investments.

Additionally, current systems rely on images instead of interactive 3D visualization, making it difficult for users to understand property layout. Investors must manually calculate investment shares and profit/loss, which reduces transparency and increases complexity.

Another limitation is the absence of analytics dashboards. Users cannot track investment performance, portfolio growth, or returns within the application. Sellers and brokers also lack tools to manage investor participation and financial records.

EstateFlow eliminates these drawbacks by introducing a Flutter-based mobile application integrated with Firebase backend services. The system provides co-investment functionality, 3D property visualization, analytics dashboards, and financial tracking. Cloud-based architecture ensures real-time updates, secure authentication, and scalable performance.

By combining mobile accessibility, cloud infrastructure, and interactive visualization, EstateFlow delivers a next-generation real estate investment platform that improves accessibility, transparency, and decision-making.

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

Requirement Analysis, or Requirement Engineering, is the process of defining user expectations for new software to be built or modified. In software engineering, it is sometimes referred to loosely by names such as requirements gathering or requirements capturing. It involves gathering, analyzing, documenting, validating, and managing the needs and conditions required to meet stakeholder expectations, ensuring alignment with project goals.

3.1 Use Case Diagram

EstateFlow System Use Case Diagram

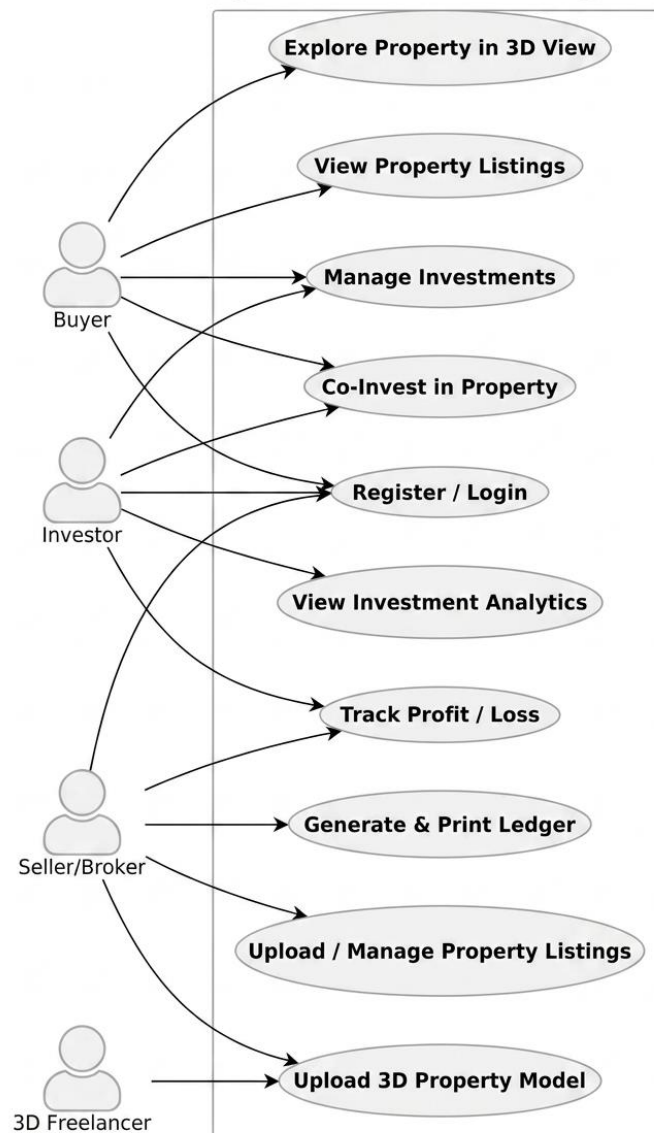


Figure 3. 1: Estate Flow Use Case

3.2 Use Case Description

Table 3. 1: Explore Property in 3D View

Use case id	UC#01
Actors	Buyer
Description	Allows buyers to interact with a 3D architectural model of a property.
Trigger	User clicks on a "View in 3D" button on a property listing.
Pre-condition	3D model data must exist for the selected property.
Post-condition	Interactive 3D viewer is loaded for the user.
Normal flow	1. User initiates 3D view; 2. System retrieves model data; 3. System renders 3D environment.
Exceptions	WebGL not supported or model fails to load.
Business rules	3D models must load within 5 seconds on standard connections.

Table 3.1 outlines this use case enables buyers to engage with immersive, interactive 3D models of properties directly within the application. Upon triggering the view, the system fetches the optimized 3D asset from cloud storage, allowing the user to rotate, zoom, and explore the architectural layout to gain a realistic spatial understanding before committing to an investment or purchase.

Table 3. 2: Manage Investments

Use case id	UC#03
Actors	Investor
Description	Enables investors to track, update, or withdraw their existing investment portfolios.
Trigger	Investor selects "My Portfolio" from the dashboard.
Pre-condition	Investor must be logged in and possess at least one active investment.
Post-condition	Updated portfolio view is presented to the investor.
Normal flow	1. Investor views list 2. Investor selects action (view/edit) 3. System updates database.
Exceptions	Invalid transaction data; server timeout during update.
Business rules	Investments cannot be deleted if there is an active co-investment lock.

Table 3.2 outlines this module provides investors with a centralized dashboard to track their active portfolios, monitor equity, and update investment details. It serves as the primary interface for users to review their financial commitments and stay informed on the status and performance of their co-invested assets in real-time.

Table 3. 3: Co-Invest in Property

Use case id	UC#04
Actors	Investor
Description	Allows multiple investors to contribute capital toward a single property.
Trigger	Investor clicks "Co-Invest" on a property listing page.
Pre-condition	Property must be flagged as "Open for Co-Investment."
Post-condition	Investment entry recorded; property equity updated.
Normal flow	1. Investor inputs amount; 2. System validates funds; 3. System confirms transaction.
Exceptions	Insufficient funds; transaction rejected by payment gateway.
Business rules	Minimum investment threshold must be met.

Table 3.3 outlines Investors use this feature to pledge capital toward properties flagged as open for co-investment. The system validates the user's available funds, executes the transaction, and updates the ownership distribution records, ensuring that all financial contributions are accurately captured and reflected across the platform's ledger.

Table 3. 4: Upload / Manage Property Listings

Use case id	UC#05
Actors	Seller/Broker
Description	Allows authorized users to add, update, or remove property listings.
Trigger	User navigates to the "Broker Dashboard" and selects "New Listing."
Pre-condition	User must have verified "Broker" or "Seller" account status.
Post-condition	Listing status updated in the public database.
Normal flow	1. User inputs property details; 2. System validates data 3. System saves to database.
Exceptions	Invalid image format; mandatory fields left empty.

Table 3.4 outlines authorized brokers and sellers use this module to perform full CRUD operations on their property listings. It provides a structured interface for submitting mandatory details and multimedia, which the system then validates and publishes to the live marketplace for potential buyers and investors to view.

Table 3. 5: Upload 3D Property Model

Use case id	UC#06
Actors	3D Freelancer
Description	Allows the freelancer to upload processed 3D files to be linked to a property.
Trigger	Freelancer selects "Upload Model" assigned to a specific project.
Pre-condition	Freelancer must be assigned to the specific property ID.
Post-condition	File is uploaded and linked to the property listing.
Normal flow	1. Freelancer uploads file 2. System performs virus/format scan 3. System stores file.
Exceptions	File size exceeds limit; incompatible file format.
Business rules	Files must be in supported formats (.obj, glb, .gltf) and under 50MB.

Table 3.5 outlines this use case enables 3D Freelancers to upload finalized property models directly to their assigned projects. The system performs an automated integrity scan to ensure file format compatibility and size compliance before storing the asset in cloud storage. Once validated, it links the model to the specific property ID, making the 3D viewing feature instantly available for potential buyers and investors.

Table 3. 6: Register / Login

Use case id	UC#07
Actors	Buyer, Investor, Seller/Broker
Description	Handles user authentication and access control to the platform.
Trigger	User clicks the "Sign Up" or "Login" button.
Pre-condition	None (or user has registered credentials).
Post-condition	User session is active; dashboard or requested page is loaded.
Normal flow	1. User provides credentials 2. System validates against database 3. System grants access.
Exceptions	Invalid credentials; account locked due to security protocols.
Business rules	Passwords must meet complexity requirements; session expires after 30 minutes of inactivity.

Table 3.6 outlines this fundamental use case handles secure user authentication, supporting multiple roles like Buyer, Investor, Broker, and Freelancer. By verifying credentials and issuing secure tokens, the system manages session state and enforces role-based access control, ensuring that every user interface is customized according to their specific permissions and account type.

Table 3. 7: View Investment Analytics

Use case id	UC#08
Actors	Investor
Description	Displays performance data and trends for the investor's current properties.
Trigger	Investor navigates to the "Analytics" tab.
Pre-condition	User must be logged in and have active investments.
Post-condition	Analytics dashboard (charts/graphs) is displayed.
Normal flow	1. System calculates current valuation/trends 2. System fetches historical data 3. Data visualized.
Exceptions	Data unavailable for selected date range; service connection error.
Business rules	Analytics must update at least once every 24 hours.

Table 3.7 outlines investors can access this module to visualize the performance of their portfolios through interactive charts and trend reports. By aggregating historical data and market values, the system presents complex financial insights in an easy-to-digest format, empowering users to make informed, data-driven decisions about their assets.

Table 3. 8: Track Profit / Loss

Use case id	UC#09
Actors	Investor, Seller/Broker
Description	Monitors financial gains or losses associated with property investments or sales.
Trigger	User selects "Financial Reports" from the user menu.
Pre-condition	User has a history of completed or active transactions.
Post-condition	Financial statement is generated and displayed.
Normal flow	1. System aggregates transaction history 2. Calculates net profit/loss 3. Displays summary.
Exceptions	Discrepancy in transaction records; server error.
Business rules	Calculation must account for initial investment, maintenance costs, and current market value.

Table 3.8 outlines this module monitors the financial outcomes of an investor's transactions, calculating net gains or losses based on initial investments and current market fluctuations. It provides an automated summary of financial health, giving users a clear and accurate view of their realized and unrealized financial performance.

Table 3. 9: Generate & Print Ledger

Use case id	UC#10
Actors	Seller/Broker
Description	Allows brokers to extract a formal financial ledger for accounting purposes.
Trigger	Broker clicks "Export/Print Ledger" in the accounting section.
Pre-condition	Broker must have authorization to access financial records.
Post-condition	A PDF or printable document is generated for the user.
Normal flow	1. System retrieves transaction records 2. Formats data into a ledger 3. Prompts download/print.
Exceptions	Empty record set; browser blocks file download.
Business rules	Ledger must be timestamped and follow standard financial reporting formats.

Table 3.9 outlines this administrative use case allows brokers to extract comprehensive financial records for a selected property or period. The system aggregates all transaction data, formats it into a professional, timestamped document, and provides an export option, ensuring that brokers have the necessary documentation for formal accounting and auditing.

3.3 Functional Requirements

Functional requirements define the specific behaviors, operations, and services that the **Estate Flow Platform** must perform to fulfill its intended objectives. These requirements describe how the system should behave under specific conditions and how users and administrators will interact with the platform to accomplish their tasks. The following subsections outline the major functionalities of the Estate Flow system.

Table 3.10: Functional Requirements of EstateFlow

Requirement ID	Requirement Name	Description
FR-01	User Authentication & Role Management	The system must provide secure multi-role registration and login using JWT-based authentication and enforce Role-Based Access Control (RBAC) to restrict users to their respective dashboards and functionalities.
FR-02	Advanced Property Search & Filtering	The system must allow users to search properties using filters such as price, location, type, and amenities, with fast response and alternative suggestions when no exact matches are found.
FR-03	Dynamic Property Listing Management	The system must enable Sellers/Brokers to perform full CRUD operations on property listings, including file validation and real-time status updates (Active, Sold, Co-Investment).
FR-04	Interactive 3D Model Rendering	The system must provide an interface to explore properties using interactive 3D models with features like zoom, rotation, and walkthrough, ensuring compatibility and smooth rendering.

FR-05	Co-Investment Contribution Workflow	The system must allow multiple investors to contribute funds to properties, validate transactions in real time, update equity distribution, and notify stakeholders.
FR-06	3D Asset Upload & Linking	The system must allow 3D Freelancers to upload compatible 3D models with validation checks and automatically link them to the respective property.
FR-07	Financial Analytics Dashboard	The system must provide investors with real-time analytics including ROI, valuation, and market trends using dynamic data visualization.
FR-08	Transactional Profit/Loss Tracking	The system must calculate and track financial performance, including profit/loss based on transactions, costs, and market changes.
FR-09	Automated Ledger Generation & Export	The system must generate professional financial ledgers for transactions with export options (PDF/Excel), including timestamps and tamper-proof features.

Table 3.10 outlines the functional requirements of the system, defining the key features and operations it must perform. Each requirement is uniquely identified and described to ensure clarity in system behavior. The table serves as a reference for development and testing, ensuring that all core functionalities are properly implemented and validated.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the Estate Flow Platform must meet to ensure a reliable, secure, and efficient user experience. These requirements describe *how* the system performs its functions rather than *what* it does. They focus on aspects like security, usability, performance, maintainability, and scalability that contribute to the system's overall quality and long-term sustainability.

3.4.1 Performance Requirements

The system must maintain high performance, ensuring that all core interactions—including search queries, filtering results, and dashboard data loads—are completed within a maximum latency of 2 seconds for 95% of requests under standard operational loads. This requirement is vital for user retention, as delays in real-time financial or property data can lead to user frustration and drop-off. The architecture will utilize efficient database indexing and caching strategies to minimize disk I/O and optimize query processing. Furthermore, frontend assets will be minified and optimized to ensure quick initial page loads, contributing to a seamless user experience across varied network conditions.

3.4.2 Scalability Requirements

EstateFlow must be architected for horizontal scalability, allowing the system to seamlessly accommodate an increasing volume of concurrent users and data without requiring significant code refactoring or infrastructure overhauls. This will be achieved by utilizing a microservices-based approach deployed within auto-scaling container environments, which can automatically spin up new instances during high-traffic events, such as peak property listing periods. The database layer must support read-replicas to distribute query load, ensuring that the system performance does not degrade as the number of active users grows. Load balancers will be configured to intelligently distribute incoming traffic across multiple server nodes, providing high availability.

3.4.3 Security Requirements

Security is paramount; therefore, all sensitive data—including user credentials, personal profile information, and financial transaction history—must be fully encrypted both in transit and at rest. We will enforce TLS 1.3 for all data communications, and use AES-256 encryption for database storage to ensure the confidentiality and integrity of proprietary and personal information. The system will implement rigorous input validation and sanitation on all API endpoints to proactively defend against common attack vectors, such as SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). Additionally, rate-limiting and account lockout policies will be strictly enforced to prevent brute-force attacks and unauthorized access attempts.

3.4.4 Usability Requirements

The EstateFlow interface must be designed with a "dark mode first" philosophy, prioritizing a visual aesthetic that is modern, professional, and reduces eye strain for power users who

spend significant time on the platform. The UI must adhere strictly to WCAG 2.1 accessibility standards to ensure that all navigation, interactive elements, and content are fully usable by individuals with visual impairments or those who rely on screen-reading technologies. We will implement a responsive, fluid design system that ensures perfect parity across desktops, tablets, and mobile devices, adapting gracefully to different screen resolutions. The information architecture will be intuitive, minimizing the cognitive load by placing essential actions, such as "Manage Investments" or "Generate Ledger," within easy reach of the user. By conducting regular usability testing and incorporating user feedback, we will continuously refine the interface to ensure that the complexity of the platform's back-end logic is hidden behind a clean, simple, and efficient front-end experience.

3.4.5 Reliability Requirements

The StayBnB system must remain stable and functional even during high traffic or minor network disruptions. It should provide 99% uptime, ensuring continuous availability for property searches, bookings, and dashboard access. Automatic retries and error-handling mechanisms should be implemented to recover from transient failures without user disruption.

3.4.6 Maintainability Requirements

The codebase must be developed following clean architecture principles, emphasizing modular, decoupled components that allow developers to maintain or replace individual modules without introducing side effects across the rest of the system. Each module must be self-contained with well-defined APIs, enabling a clear separation of concerns that facilitates efficient debugging and easier unit testing. Comprehensive documentation is non-negotiable; this includes auto-generated API specifications (e.g., Swagger/OpenAPI) and up-to-date UML diagrams (use case, class, and sequence) that reflect the current state of the architecture. Coding standards and style guides will be strictly enforced through automated linting and code review processes to ensure consistency across the entire development team.

3.4.7 Interoperability and API Integration

EstateFlow must provide a robust, well-documented set of RESTful APIs, enabling the platform to integrate seamlessly with third-party service providers, such as payment gateways, real-time market data aggregators, or external CRM platforms. These APIs will

use standard JSON data formats to ensure compatibility with various front-end frameworks and external service architectures. We will implement versioning for all API endpoints, ensuring that updates to the platform do not break existing integrations for our partners. Comprehensive error handling and clear response codes will be provided to assist developers in quickly troubleshooting integration issues.

3.4.8 Data Integrity Requirements

Data stored within the system, such as user profiles, property details must remain consistent and error-free. The platform should implement validation at both the client and server sides to prevent duplication or missing entries. Backup mechanisms should be in place to restore data in case of failure, ensuring no permanent loss of information.

3.4.9 Availability Requirements

The system is required to guarantee a minimum annual availability of 99.9%, ensuring that users can access their portfolios, property listings, and financial tools whenever they need to make critical decisions. This commitment to uptime necessitates a resilient infrastructure that includes redundant server components and multi-region deployment strategies to mitigate the impact of localized data center outages. While scheduled maintenance may occur for security updates or feature deployments, it must be performed during off-peak hours and communicated to all platform users at least 48 hours in advance.

3.4.10 Extensibility Requirements

The architecture should allow for future enhancements without requiring major changes to the core system. The modular design should support easy integration of additional AI models, new features (like voice search or VR tours), and third-party APIs. This ensures that the platform remains adaptable to future technological advancements.

3.4.12 Compliance and Legal Requirements

The system must be fully compliant with major international data protection regulations, such as GDPR and CCPA, which mandate how we collect, process, and store user information. This includes building specific features into the platform that give users direct control over their data, such as the ability to download an archive of their personal information or request the complete deletion of their account records upon closure. We will maintain detailed audit logs of data access and modification, which are essential for

fulfilling regulatory reporting requirements and demonstrating that we are handling sensitive user data in a transparent and responsible manner.

3.4.13 Accessibility Requirements

The system should ensure accessibility for all users, including those with disabilities. Features such as text alternatives for images, keyboard navigation, and color-contrast adherence must be implemented. The layout must remain clear and readable, enhancing the overall inclusivity of the EstateFlow experience.

3.4.14 Load Handling Requirements

The system should handle concurrent access from a minimum of 500 users simultaneously without noticeable delay or service degradation. Performance testing tools such as Postman or Apache JMeter can be used to verify load performance during the deployment phase.

3.4.15 Audit and Logging Requirements

The platform should maintain comprehensive logs of all major activities including user logins, property additions, edits, and deletions. These logs should be securely stored for administrative review and serve as a diagnostic tool for identifying potential issues or unauthorized access attempts.

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

This chapter describes the overall design and architecture of the EstateFlow mobile application. The system is designed using modular architecture to support scalability, maintainability, and efficient interaction between users and Firebase backend services. The design includes system architecture, data representation, process flow, activity diagrams, sequence diagrams, and class diagram. These components collectively define the structural and behavioral aspects of the EstateFlow system.

The EstateFlow application is developed using Flutter and Dart for the mobile interface, while Firebase services are used for authentication, database management, and storage. The architecture supports multiple user roles including Buyer, Investor, Seller/Broker, and 3D Freelancer. Each role interacts with the system through dedicated modules such as property management, co-investment, analytics, ledger generation, and 3D visualization.

4.1 System Architecture

The EstateFlow system follows a layered mobile application architecture consisting of presentation layer, application layer, and cloud backend layer. The presentation layer is developed using Flutter and provides role-based interfaces for buyers, investors, sellers, and freelancers. This layer handles user interaction and displays data received from backend services.

The application layer contains the core modules including authentication module, property management module, investment module, analytics module, profit/loss module, ledger module, and 3D viewer module. These modules process user requests and communicate with Firebase backend services.

The backend layer is implemented using Firebase services. Firebase Authentication manages user login and registration. Cloud Firestore database stores users, properties, investments, analytics, and ledger records. Firebase Cloud Storage stores property images and 3D models. The 3D viewer module retrieves models from cloud storage and renders them in the mobile application.

This architecture ensures real-time data synchronization, secure authentication, and scalable performance. The modular design allows independent development and maintenance of each component.

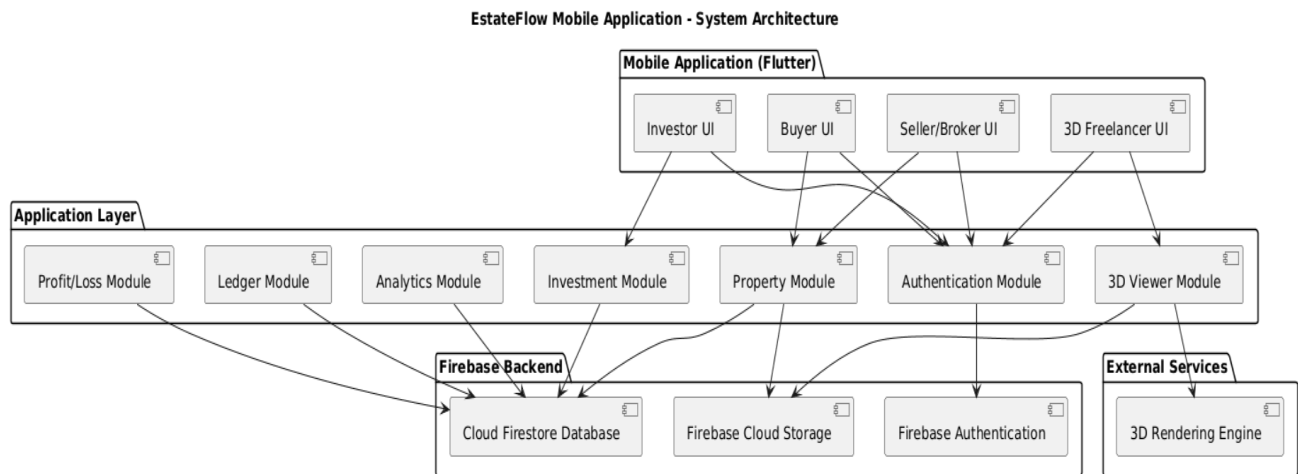


Figure 4. 1: EstateFlow System architecture

The architecture diagram shows four user interfaces including Buyer UI, Investor UI, Seller/Broker UI, and 3D Freelancer UI. These interfaces interact with application modules. The authentication module connects to Firebase Authentication for secure login. The property module stores property data in Firestore and images in Cloud Storage. The investment module records shared investments. The analytics and profit/loss modules calculate financial performance. The ledger module generates transaction records. The 3D viewer module retrieves models from storage and renders them using Flutter-compatible 3D rendering libraries.

4.2 Data Representation

Data representation in EstateFlow is designed using structured entities stored in Cloud Firestore. The database is organized into collections such as users, properties, investments, analytics, ledger, and 3D models. Each collection contains documents representing individual records.

User data includes user ID, name, email, role, and authentication information. Property data contains property ID, title, location, price, description, seller ID, and 3D model reference. Investment data stores investor ID, property ID, investment amount, and ownership percentage. Analytics data stores calculated returns and performance metrics. Ledger data stores transaction history. 3D model data stores file path and property mapping.

This structured representation ensures efficient data retrieval and real-time synchronization between mobile app and cloud database.

4.2.1 Key Entities and Relationships

The key entities in the EstateFlow system include User, Property, Investment, 3D Model, Analytics, Profit/Loss, and Ledger, each playing a critical role in the overall functionality of the platform. The User entity represents all individuals interacting with the system, including buyers, investors, sellers/brokers, and 3D freelancers. Each user type has specific roles and permissions, enabling them to perform actions relevant to their responsibilities within the platform.

The Property entity represents real estate listings created and managed by sellers or brokers. These listings contain detailed information such as location, price, property type, and associated media files. The Investment entity captures the relationship between investors and properties, enabling a co-investment model. In this structure, a single property can have multiple investors, and each investor can participate in multiple properties, forming a many-to-many relationship.

The 3D Model entity is directly linked to properties and stores metadata related to interactive 3D assets. These models enhance user experience by allowing virtual exploration of properties. The Analytics entity processes data derived from investments and property performance to generate insights such as return on investment (ROI), valuation trends, and market analysis.

4.2.2 ER Diagram

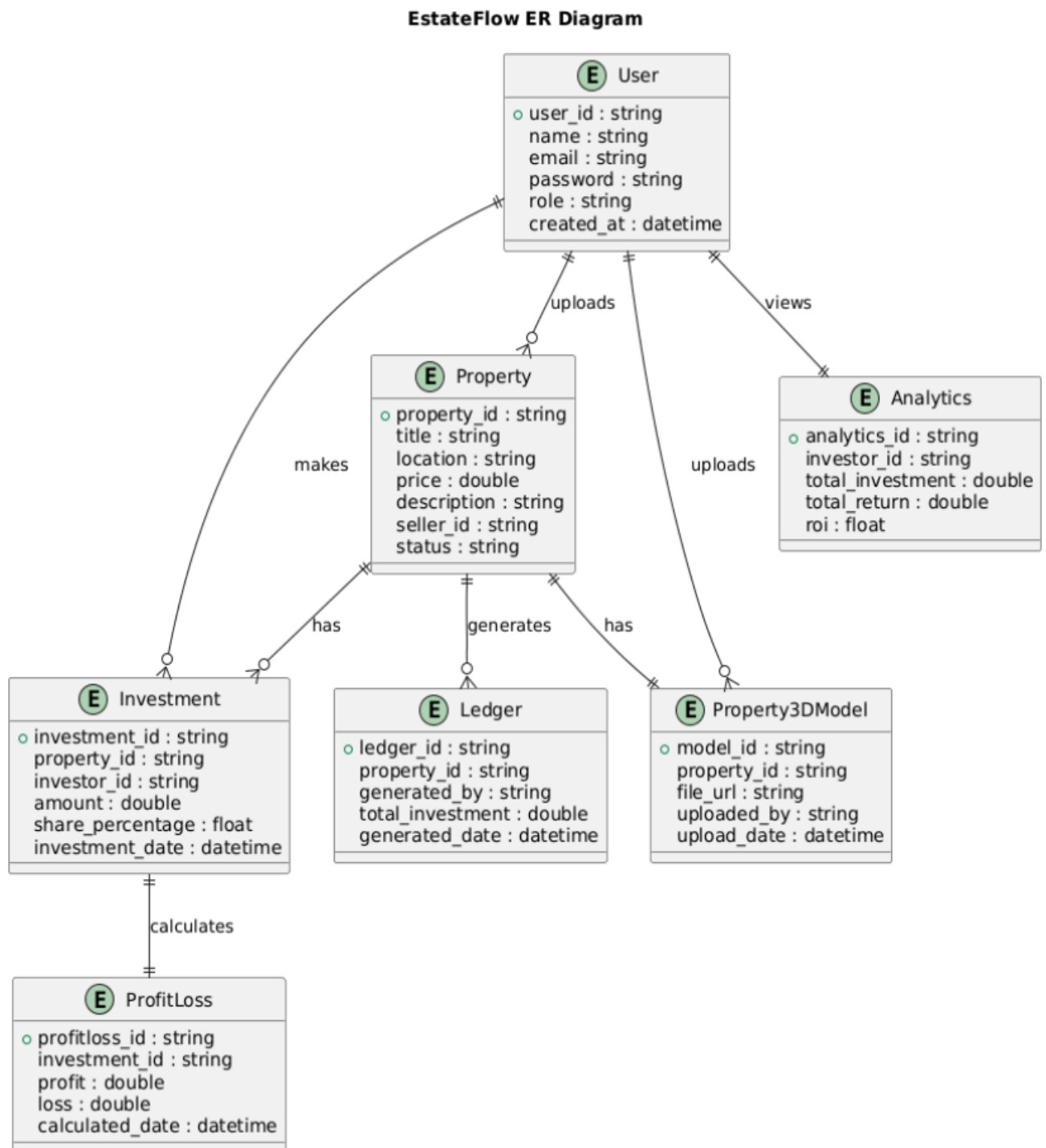


Figure 4. 2: EstateFlow ER Diagram

The ER diagram of EstateFlow represents the data structure and relationships between system entities. The main entity in the system is User, which represents buyers, investors, sellers, and 3D freelancers. Each user has attributes such as user ID, name, email, password, role, and creation date.

The Property entity stores real estate listings uploaded by sellers. Each property includes property ID, title, location, price, description, seller ID, and status. A seller can upload multiple properties, establishing a one-to-many relationship between User and Property.

The Investment entity represents co-investment records. It stores investment ID, property ID, investor ID, amount, share percentage, and date. A property can have multiple investments, and an investor can invest in multiple properties, creating many-to-one relationships.

The Property3DModel entity stores 3D models uploaded by freelancers. Each property can have one associated 3D model. The model contains file URL and upload details.

The Analytics entity stores calculated investment performance for investors. It includes total investment, total return, and ROI values.

The ProfitLoss entity calculates financial results for each investment. It stores profit, loss, and calculation date.

The Ledger entity stores financial transaction summaries generated by sellers. It contains property ID, total investment, and generated date.

These entities and relationships define the database structure of the EstateFlow mobile application and support co-investment, 3D visualization, analytics, and financial tracking features

4.3 Process Flow/Representation

The process flow of EstateFlow describes how users interact with the system to perform operations such as login, property upload, co-investment, and 3D visualization. The activity

diagrams illustrate step-by-step workflows for these processes. Each activity diagram shows user actions, system processing, and decision points.

4.3.1 Activity Diagrams

Activity diagrams represent workflow of system processes. They show sequential flow of activities, conditions, and system responses. The diagrams help understand how users interact with the EstateFlow mobile application.

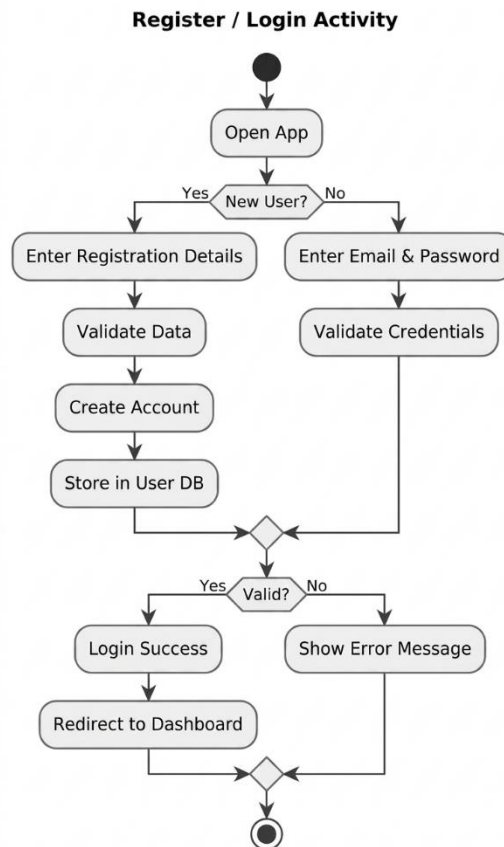


Figure 4. 3: User Login and Registration Activity Diagram

The User Login and Registration activity diagram illustrates the authentication process. The workflow begins when the user opens the mobile application. The user selects register or login option. If the user is new, registration details are entered including email and password. The system validates the input and creates a new account using Firebase Authentication.

If the user selects login, credentials are entered and validated. The system checks authentication through Firebase. If credentials are valid, the user is redirected to dashboard. If credentials are invalid, an error message is displayed. This process ensures secure user authentication and access control.

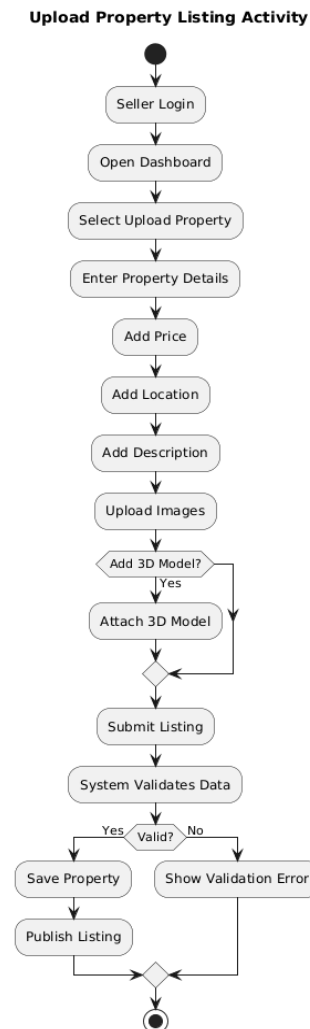


Figure 4. 4: Upload Property Listing (Seller/Broker) Activity Diagram

This activity diagram shows the workflow for uploading property listings. The process begins when seller logs into the application. The seller selects upload property option and enters property details including title, price, location, description, and images.

The seller may also attach 3D model if available. The system validates input fields and stores data in Firestore database. After successful validation, property is published and becomes visible to buyers and investors. If validation fails, error message is displayed. This workflow ensures structured property listing management.

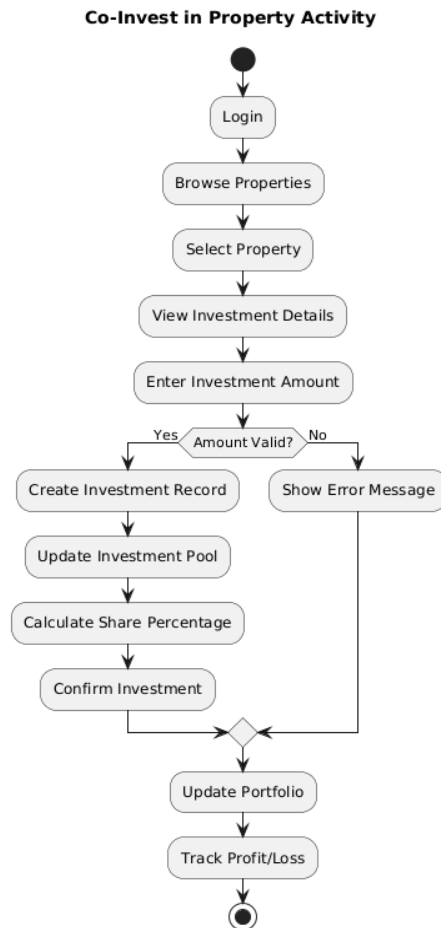


Figure 4. 5: Co-Invest in Property (Buyer/Investor) Activity Diagram

The Co-Invest activity diagram describes the shared investment process. The buyer or investor logs into the system and browses property listings. After selecting a property, investment details are displayed. The user enters investment amount.

The system validates the amount and calculates ownership share. Investment record is stored in Firestore database. Portfolio is updated and profit/loss tracking is enabled. If invalid amount is entered, error message is displayed. This process supports fractional ownership of properties.

Upload & Explore 3D Model Activity

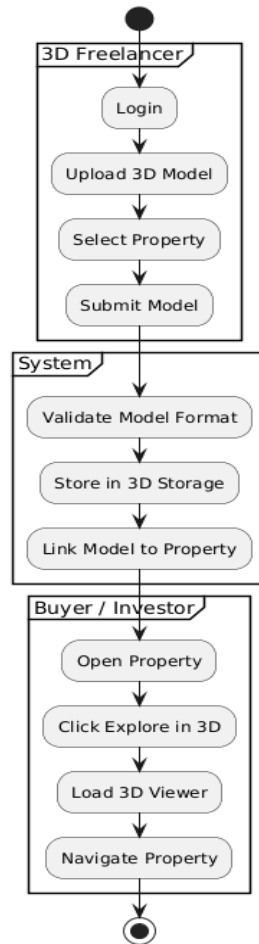


Figure 4. 6: Upload & Explore 3D Property Model Activity Diagram

The Upload & Explore 3D Property Model activity diagram illustrates the workflow involving two main actors: the 3D Freelancer and the Buyer/Investor. The process begins when the 3D freelancer logs into the EstateFlow mobile application and navigates to the upload 3D model

section. The freelancer selects a property for which the 3D model is to be uploaded and chooses the model file from the device. The system then validates the file format, size, and compatibility with Flutter-based 3D visualization libraries. If the file is valid, the model is uploaded to Firebase Cloud Storage. After successful upload, the system generates a file URL and stores the model reference in Cloud Firestore, linking it to the selected property.

4.4 Design Models

Design models describe the internal behavior and interaction of system components in EstateFlow. These models illustrate how users communicate with system modules and how data flows between the mobile application and Firebase backend. The design models include sequence diagrams and class diagram, which collectively define system behavior and structure. Sequence diagrams show the order of interactions between actors, UI, modules, and backend services, while class diagram defines entities and their relationships. These models help in understanding runtime communication and object-oriented structure of the EstateFlow system.

4.4.1 Sequence Diagram

Sequence Diagrams are a type of interaction diagram used to illustrate how different objects or components in a system communicate with each other to perform a specific operation. They focus on the dynamic behavior of the system by showing the sequence of messages exchanged between entities during a particular process or use case.

These diagrams are time-oriented, meaning the progression of time is represented vertically from top to bottom. Each object involved in the interaction is depicted along the horizontal axis, while the messages passed between them are shown as arrows. This visual structure makes it easy to understand not only which components are involved, but also the exact order in which interactions occur.

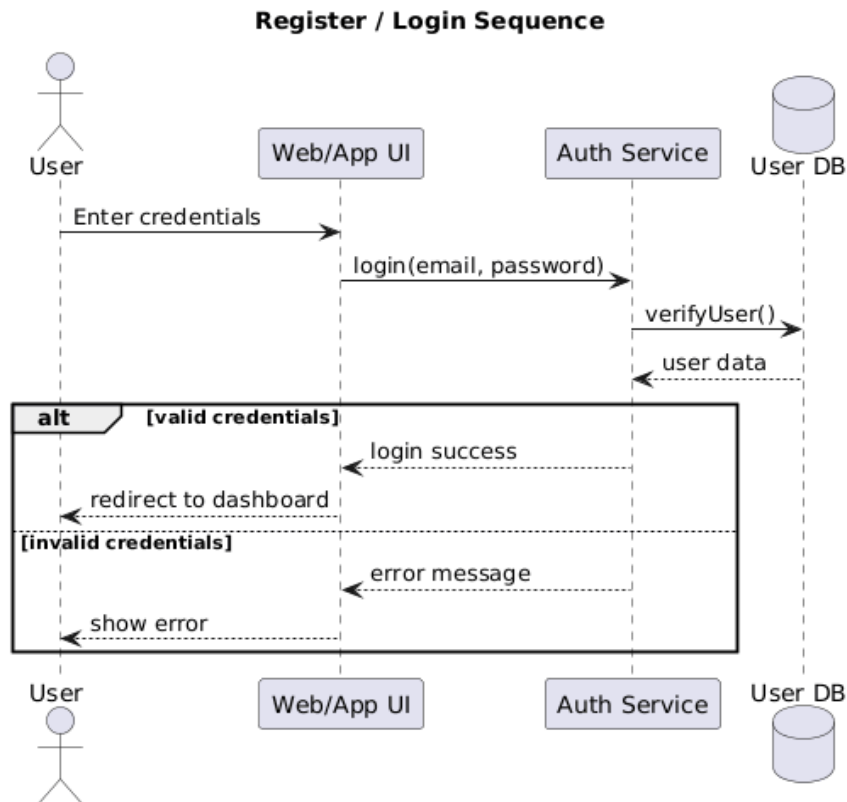


Figure 4. 7: Sequence Diagram for User Registration and Login

The User Login and Registration sequence diagram illustrates the interaction between the user, mobile UI, authentication module, and Firebase Authentication service. The process begins when the user opens the EstateFlow mobile application and enters login or registration details. The mobile UI sends the credentials to the authentication module for validation. The authentication module then forwards the request to Firebase Authentication service. Firebase verifies the user credentials or creates a new account in case of registration. After verification, Firebase returns a success or failure response to the authentication module. The authentication module processes the response and sends the result back to the mobile UI. If authentication is successful, the system redirects the user to the dashboard. If authentication fails, an error message is displayed. This sequence

ensures secure login, role-based access, and real-time authentication using Firebase services.

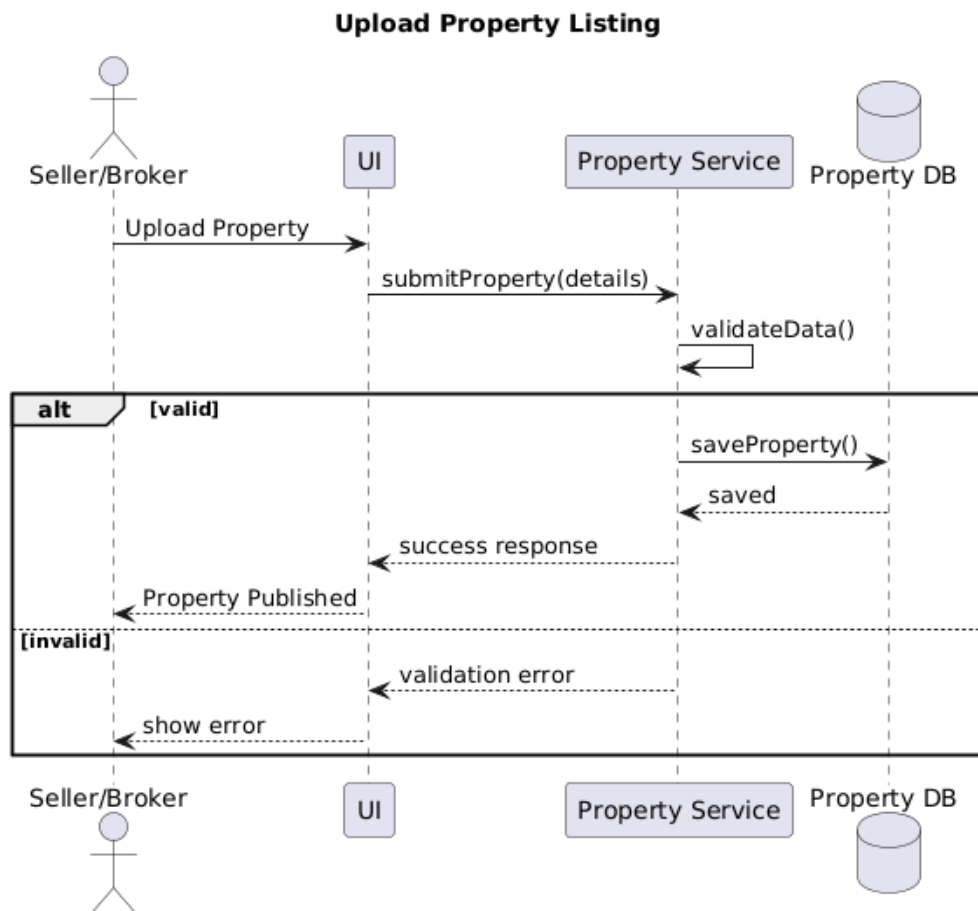


Figure 4. 8: Upload Property Listing (Seller/Broker) Sequence Diagram

The Upload Property Listing sequence diagram describes the interaction between the seller, mobile interface, property module, Firestore database, and cloud storage. The process starts when the seller logs into the EstateFlow application and selects the upload property option. The seller enters property details including title, location, price, and description. The mobile UI sends the entered data to the property module for validation. The property module verifies the input fields and prepares the data for storage. Property images and

optional 3D model files are uploaded to Firebase Cloud Storage. After successful upload, the property metadata is stored in Cloud Firestore database.

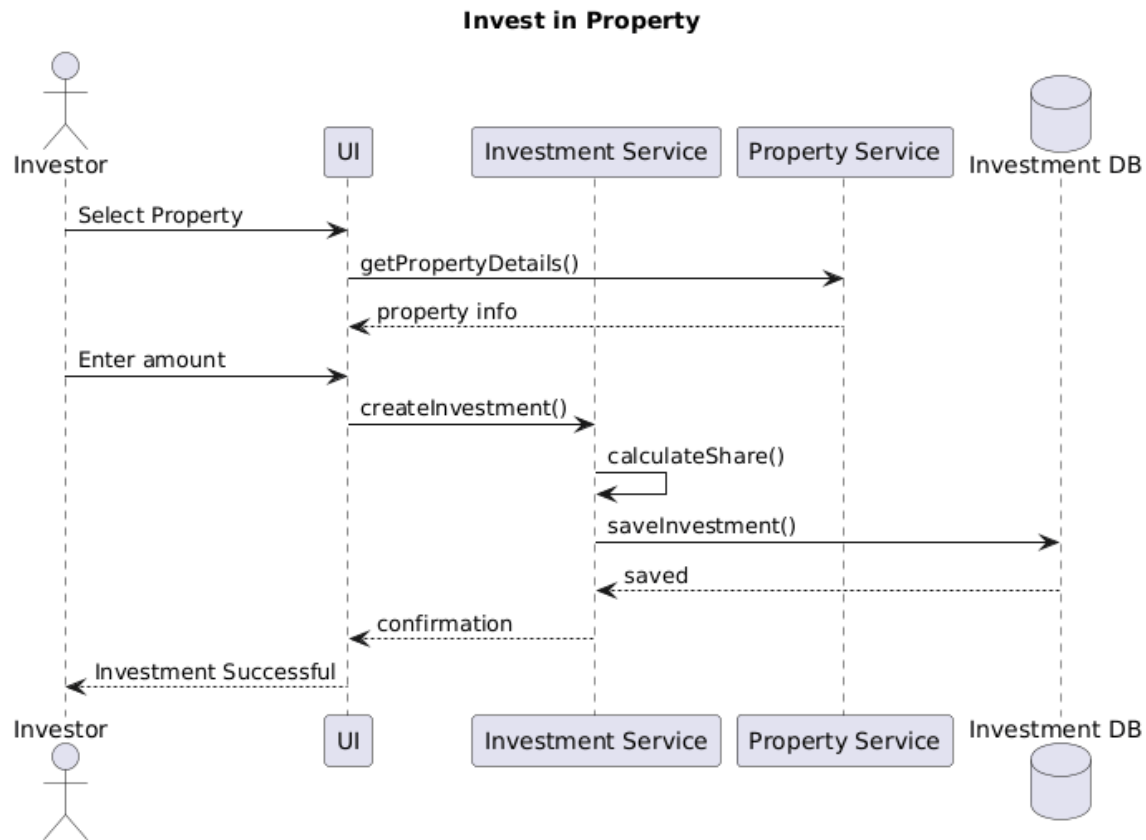


Figure 4. 9: Co-Invest in Property (Buyer/Investor) Sequence Diagram

The Co-Invest sequence diagram shows how buyers or investors invest in properties. The process begins when a user selects a property from the listing screen. The mobile UI requests property details from Firestore database. The system returns property information including price and available investment shares. The investor enters the desired investment amount through the UI. The request is sent to the investment module for processing. The investment module validates the entered amount and calculates ownership percentage. The calculated data is then stored in Firestore database as a new investment record. The database confirms the successful storage of the investment.

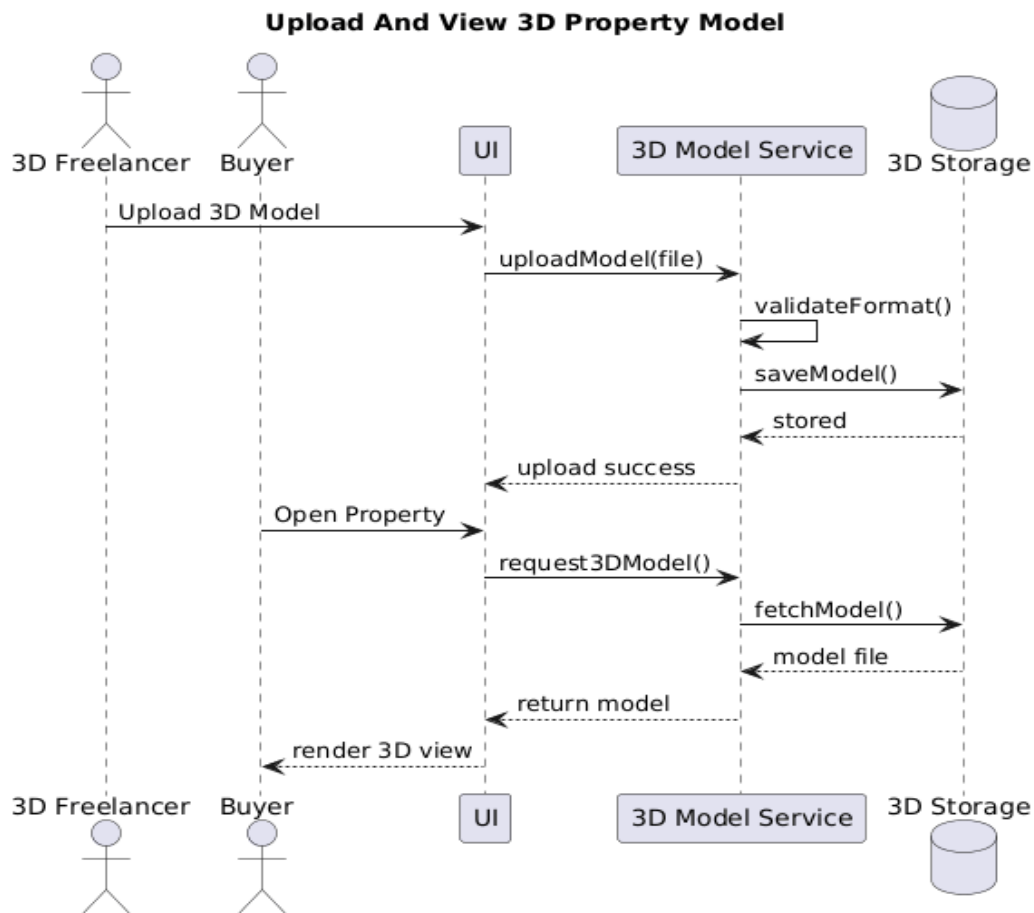


Figure 4. 10: Upload & Explore 3D Property Model Sequence Diagram

This sequence diagram illustrates how 3D freelancers upload models and how users view them. The process begins when a 3D freelancer logs into the system and selects upload model option. The freelancer selects a property and uploads a 3D model file. The mobile UI sends the file to the 3D module. The 3D module validates file format and sends it to Firebase Cloud Storage. After successful upload, the file URL is returned to the 3D module. The module stores model reference in Firestore database and links it to the property. When a buyer or investor selects explore 3D option, the UI requests model data from database.

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize, document, and construct executable code. Class diagrams describe the static view of an application and are crucial for modeling system functionalities and collaborations.

4.4.2.1 Class Diagram

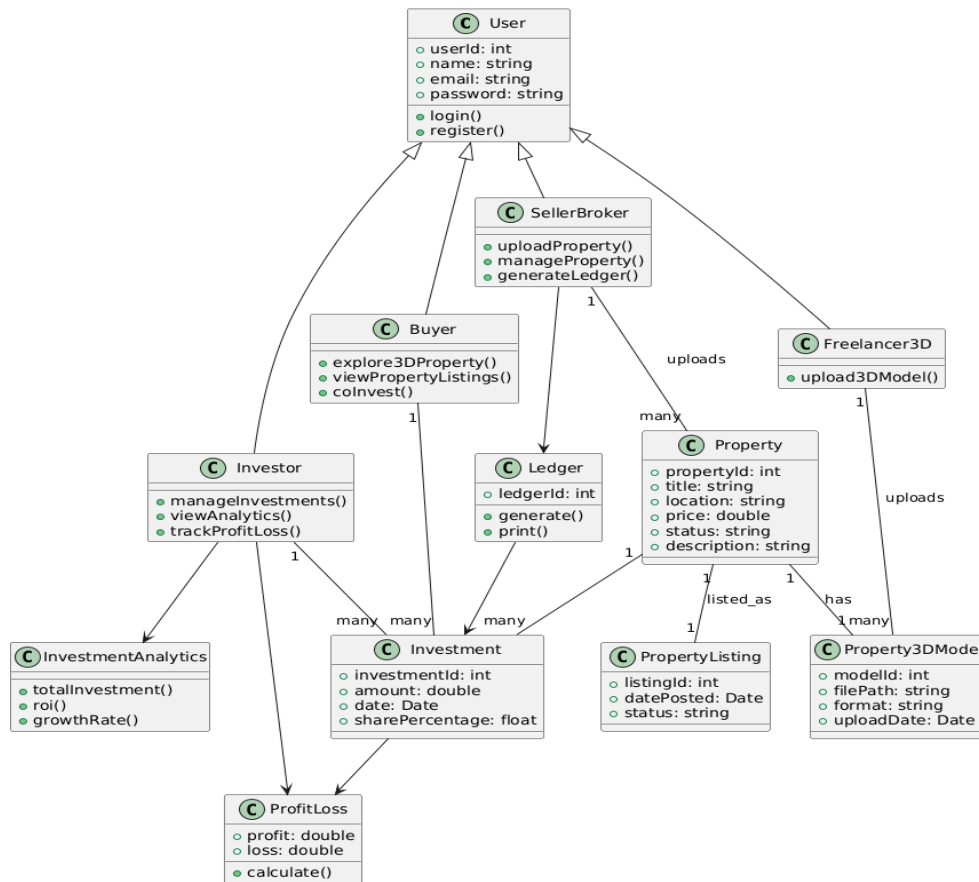


Figure 4. 11: Class Diagram

The Class Diagram of EstateFlow represents the structural design of the system and relationships between core entities. The main class in the system is User, which contains attributes such as user ID, name, email, password, and role. The User class is generalized into

Buyer, Investor, Seller/Broker, and 3D Freelancer roles. The Property class stores information about real estate listings including title, location, price, description, and seller ID. The Seller/Broker class is associated with the Property class to upload and manage listings. The Investment class represents shared ownership and stores investment amount, investor ID, and share percentage. The Buyer and Investor classes interact with the Investment class for co-investment. The Property3DModel class stores 3D model information uploaded by freelancers. The Analytics class calculates investment performance and ROI values. The ProfitLoss class calculates financial gains and losses. The Ledger class maintains transaction records and investment summaries. These classes interact to form a modular and scalable system architecture.

Chapter 5

IMPLEMENTATION

5 Implementation

This chapter describes the implementation details of the EstateFlow mobile application. The system is implemented using Flutter framework with Dart programming language, while Firebase services are used for authentication, database management, and cloud storage. The implementation includes algorithms for core functionalities such as user authentication, property upload, co-investment, analytics generation, ledger creation, and 3D model visualization.

The mobile application communicates with Firebase backend services in real time. Firebase Authentication handles login and registration. Cloud Firestore stores structured data including users, properties, investments, and analytics. Firebase Cloud Storage stores images and 3D property models. The Flutter-based UI renders data and provides interactive user experience.

5.1 Algorithm

The following algorithms describe major functionalities implemented in EstateFlow mobile application.

5.1.1 User authentication

Function: authenticateUser (email, password)

Input:

- email (string)
- password (string)

Output:

user (object) or error (string)

Description:

This algorithm handles secure user authentication for EstateFlow mobile application. It uses

Firebase Authentication to verify user credentials and grant access to system features. The algorithm validates input fields and checks credentials from Firebase authentication service.

Pseudocode:

- User enters email and password
- Send authentication request to Firebase Authentication
- If user exists
 - a. Verify password
 - b. If valid, retrieve user role from Firestore
 - c. Redirect user to dashboard
- Else
 - Return "Invalid credentials"
- If network error
 - Return "Authentication failed"

5.1.2 Upload Property Listing

Function: uploadProperty (propertyData)

Input:

- property title
- location
- price
- description
- images

Output:

- property object or error

Description:

This algorithm allows seller or broker to upload property listing.

Pseudocode:

- Seller enters property details
- Validate required fields
- Upload images to Firebase Storage
- Get image URLs
- Create property object
- Store property in Firestore
- Return success message

5.1.3 Co-Invest in Property

Function: investProperty (userId, propertyId, amount)

Input:

- userId
- propertyId
- amount

Output:

- investment record

Description:

This algorithm allows buyers and investors to co-invest in property.

Pseudocode:

- User selects property
- Enter investment amount
- Validate minimum amount
- Retrieve property price
- Calculate ownership share

- Create investment record
- Store in Firestore
- Update user portfolio

5.1.4 Upload 3D Property Model

Function: upload3DModel (propertyId, modelFile)

Input:

- propertyId
- model file

Output:

- model URL

Description:

This algorithm uploads 3D model for property visualization.

Pseudocode

- Freelancer selects property
- Upload model file
- Validate file format
- Upload to Firebase Storage
- Get model URL
- Save model reference in Firestore
- Link model to property

5.1.5 View Investment Analytics

Function: generateAnalytics (userId)

Input:

- userId (string)

Output:

- analytics object

Description:

This algorithm generates investment analytics.

Pseudocode

- Retrieve user investments
- Calculate total investment
- Calculate returns
- Calculate ROI
- Create analytics object
- Return analytics data

5.1.6 Track Profit / Loss

Function: calculateProfitLoss (investmentId)

Input:

- investmentId

Output:

- profit/loss object

Description:

This algorithm calculates profit or loss for investment.

Pseudocode:

- Retrieve investment record

- Retrieve property value
- Calculate current value
- Compare with invested amount
- Calculate profit or loss
- Return result

5.1.7 Generate Ledger

Function: generateLedger (propertyId)

Input:

- propertyId

Output:

- ledger report

Description:

This algorithm generates financial ledger for property.

Pseudocode:

- Retrieve all investments
- Calculate total investment
- Collect investor details
- Create ledger object
- Store in Firestore
- Return ledger

5.2 External APIs

EstateFlow integrates a variety of external APIs, libraries, and cloud-based services to enhance its functionality, scalability, and performance. These integrations support critical

features such as secure user authentication, efficient data storage, and high-quality 3D visualization of properties.

Table 5. 1: External APIs

API / Service	Purpose	Usage
Firebase Authentication	User login & registration	Secure authentication
Cloud Firestore	Database	Store users, properties, investments
Firebase Cloud Storage	File storage	Store images and 3D models
Flutter 3D Viewer Library	3D visualization	Render property models
Firebase SDK	Backend integration	Connect Flutter with Firebase
Google Play Services	Mobile deployment	Android support

Table 5.1: The External APIs table lists services used in EstateFlow mobile application. Firebase Authentication is used for secure login and registration. Cloud Firestore stores structured data including users, properties, and investments. Firebase Cloud Storage stores images and 3D models uploaded by sellers and freelancers. Flutter-compatible 3D viewer library renders interactive property models inside mobile application. Firebase SDK connects Flutter application with backend services. Google Play Services support deployment and mobile integration.

These APIs enable scalable cloud architecture and interactive user experience.

5.3 User Interface

The EstateFlow mobile application provides a user-friendly interface developed using Flutter. The UI is designed to support multiple user roles including buyer, investor, seller, and 3D freelancer. Each user accesses role-based dashboard after login.

The login and registration screen allows users to authenticate securely. The dashboard screen displays property listings and navigation options. Buyers and investors can browse properties and explore them in 3D view. Investment screen allows users to enter investment amount and confirm co-investment.

5.3.1 Sign-up Screen

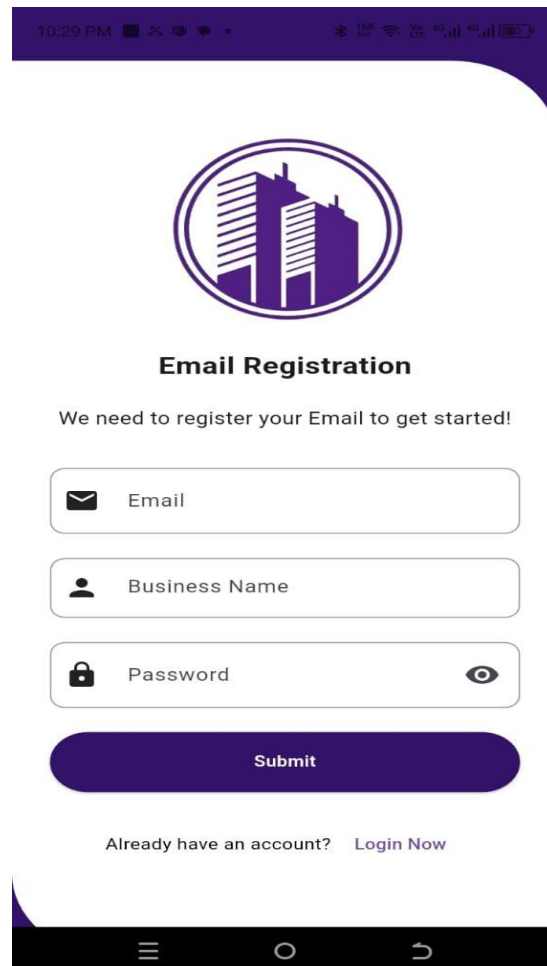


Figure 5. 1: Sign-up Screen

Figure 5.1: The Sign-up screen of the platform serves as the primary gateway for new users to establish their professional presence within the system. This interface is designed with a clean, minimalist aesthetic that utilizes a purple and white color scheme to convey a sense of security

and modern professionalism. Users are prompted to provide essential credentials, including their email address, business name, and a secure password, ensuring a personalized setup from the very first interaction.

5.3.2 Login Screen

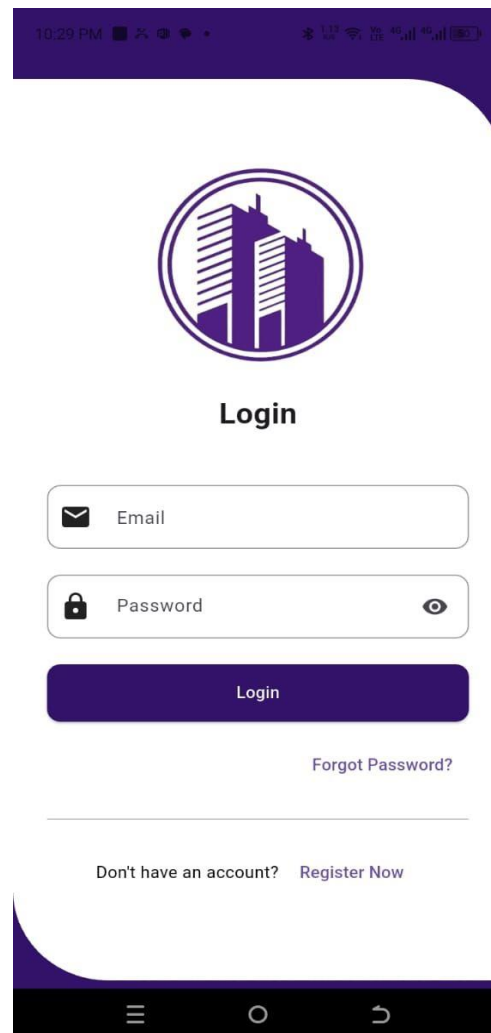


Figure 5. 2: Login Screen

Figure 5.2: The Login screen of the platform functions as the secure entry point for existing users, providing a streamlined interface to access their personalized dashboard. The layout is

optimized for efficiency, requiring only the user's registered email and password to authenticate their identity.

5.3.3 Dashboard

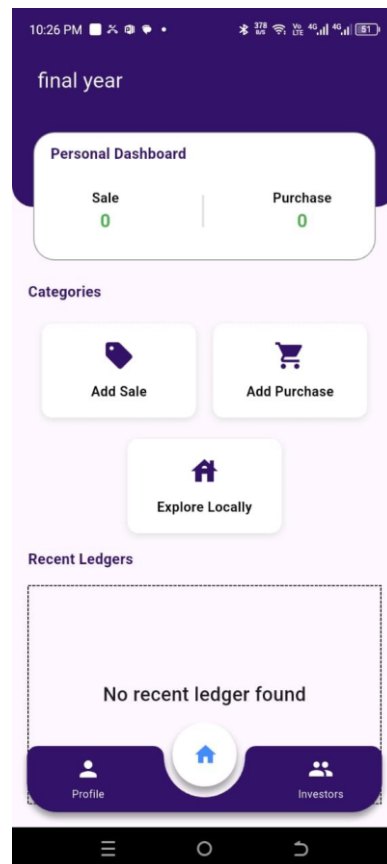


Figure 5. 3:Dashboard Screen

Figure 5.3: The Personal Dashboard screen of the platform serves as a centralized command center, providing users with a high-level overview of their real estate activities and financial metrics. At the top, a concise summary card displays real-time data for "Sale" and "Purchase" counts, allowing investors and sellers to track their transaction volume at a glance.

5.3.4 Profile Manage Screen

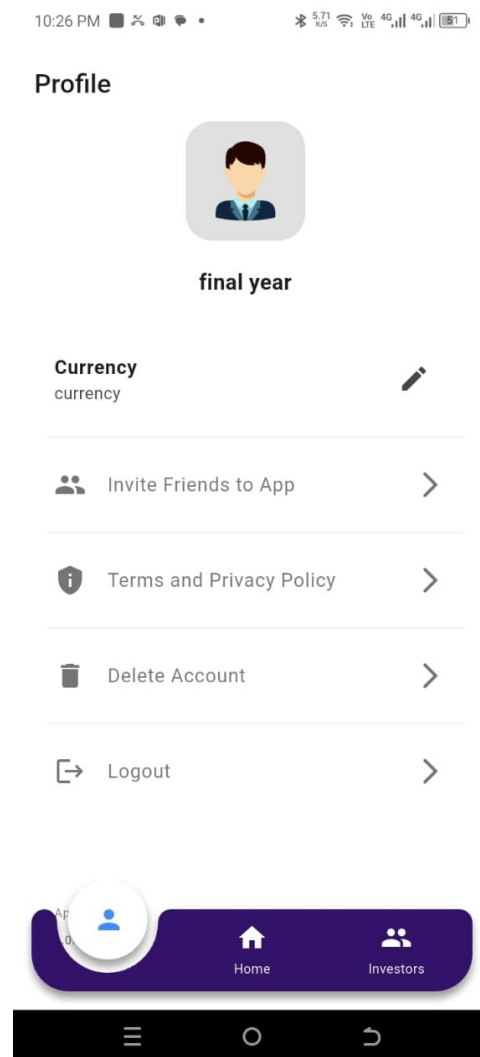


Figure 5. 4: Profile Screen

Figure 5.4: The Profile screen of the platform provides a comprehensive personal management hub where users can customize their experience and manage account-level settings. At the top of the interface, a personalized avatar and user display name establish a clear sense of identity within the application environment. This clean, list-based layout ensures that administrative tasks are accessible and uncomplicated, maintaining the platform's professional aesthetic.

5.3.5 Add Sale Property Screen

10:27 PM 211 B/S Vo LTE 4G 4G 50

← Add Sale Property Save

Enter price

Type of Property:

Agriculture

Village or City: Is it road touching?

Select Location Select Yes/No

How many acres?

Enter acres

Add Images

+

Upload 3D Model Hire Expert

Add Note

Type your description or note here...

RIGHT OVERFLOWED BY 0.749 PIXELS

Figure 5. 5: Add Sale Property Screen

Figure 5.5: The Add Sale Property screen represents a sophisticated data-entry interface designed to bridge the gap between traditional real estate listings and high-tech immersive visualization. This module allows sellers to categorize their assets with high specificity, using dynamic dropdowns to select property types like "Agriculture" and input critical geographical data such as location and road accessibility.

5.3.6 Add Purchase Property Screen

Figure 5. 6: Add Purchase Property Screen

Figure 5.6: The Add Purchase Property screen functions as a comprehensive data-entry portal designed to streamline the documentation of new real estate acquisitions within the platform. This interface allows users to categorize the nature of the purchase by selecting between "Customer," "Self," or "Investor" roles, ensuring that the transaction is recorded with the correct ownership context.

5.3.7 Available Property Screen

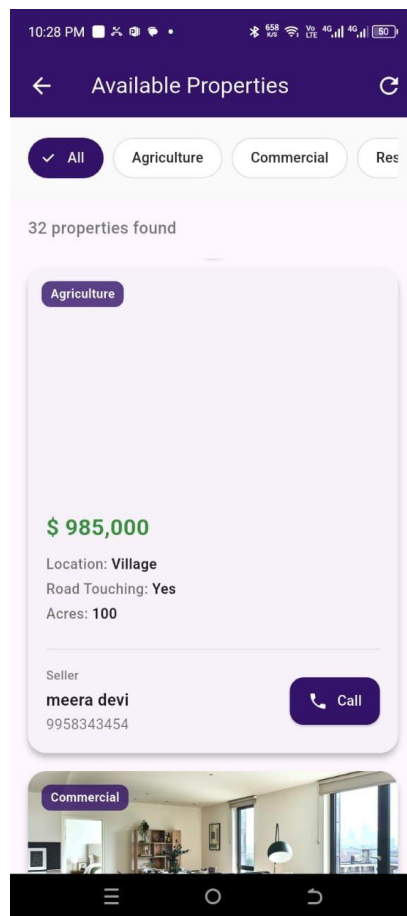


Figure 5. 7: Available Property Screen

Figure 5.7: The Available Properties screen of the platform functions as a dynamic marketplace, allowing users to browse through a diverse range of real estate listings with ease. This streamlined approach ensures that buyers and investors can navigate the market efficiently while having all critical decision-making data at their fingertips. Overall, the screen serves as a robust tool for connecting serious investors with prime property listings across various sectors.

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

Testing and evaluation are essential phases in the development of the EstateFlow mobile application to ensure system reliability, functionality, and performance. The testing process verifies that all modules including authentication, property management, co-investment, analytics, ledger generation, and 3D visualization operate correctly. Since EstateFlow is a Flutter-based mobile application integrated with Firebase backend services, testing was performed to validate both frontend UI behavior and backend cloud interactions.

Manual testing techniques were used to evaluate system workflows such as login, property upload, co-investment, and 3D model visualization. Each module was tested individually and then tested collectively through integration testing. Functional testing ensured that all user roles including Buyer, Investor, Seller/Broker, and 3D Freelancer perform their operations correctly. Unit testing verified individual functions such as authentication, investment calculation, and model loading. System testing validated the complete application workflow.

Testing was conducted using Android emulator and real mobile devices to ensure consistent performance. Firebase services were also tested for real-time data synchronization. The testing phase ensured that EstateFlow meets functional requirements, maintains data integrity, and provides smooth user interaction.

6.1 Manual Testing

Manual testing was performed to verify the behavior of the EstateFlow mobile application under different scenarios. Test cases were executed manually by interacting with the mobile interface. Each user role was tested independently to confirm that all features work correctly. Manual testing focused on login functionality, property listing upload, co-investment process, analytics generation, and 3D property visualization.

The testing also verified navigation between screens, data validation, error handling, and Firebase integration. Various input values were tested to ensure system stability. Edge cases such as invalid credentials, empty fields, and network failures were also evaluated. The results

confirmed that the EstateFlow application performs correctly under normal and exceptional conditions.

6.1.1 System Testing

System testing was performed to validate the entire EstateFlow application as a complete system. All modules were integrated and tested together. The testing ensured that users can register, login, upload properties, invest in properties, view analytics, and explore 3D models without errors.

Table 6. 1: System Testing Table

Test ID	Case	Test Scenario	Input	Expected Output	Result
ST-01		User Registration	Valid user data	Account created successfully	Pass
ST-02		User Login	Valid email/password	Dashboard displayed	Pass
ST-03		Upload Property	Property details	Property added	Pass
ST-04		Co-Invest Property	Investment amount	Investment saved	Pass
ST-05		Upload 3D Model	Model file	Model uploaded	Pass
ST-06		View 3D Model	Select property	3D viewer opens	Pass
ST-07		View Analytics	User investments	Analytics displayed	Pass
ST-08		Generate Ledger	Property selected	Ledger generated	Pass

Table 6.1 presents the system testing scenarios conducted to validate the core functionalities of the EstateFlow platform. Each test case includes a specific scenario, input conditions, and the expected system output to ensure proper behavior.

6.1.2 Unit Testing

Unit testing was performed to validate individual functions of the EstateFlow application. Each function was tested independently to ensure correct output. The tests focused on authentication logic, property upload validation, investment calculation, analytics generation, and 3D model loading.

Table 6. 2: Unit Function Table

Unit Function	Input	Expected Output	Result
authenticateUser ()	email, password	user object	Pass
registerUser ()	user data	success message	Pass
uploadProperty ()	property details	property saved	Pass
investProperty ()	amount	investment record	Pass
upload3DModel ()	model file	model URL	Pass
generateAnalytics ()	user id	analytics data	Pass
calculateProfitLoss()	investment id	profit/loss	Pass

generateLedger ()	property id	ledger report	Pass
-------------------	-------------	---------------	------

Table 6.2 presents the unit-level testing of core system functions within the EstateFlow platform. Each unit function is tested independently by providing specific inputs and verifying the expected outputs. The results indicate whether each function performs correctly according to its intended logic. This table ensures that individual components of the system are reliable and function as expected before integration into the complete system.

6.1.3 Functional Testing

Functional testing verified that all system features operate according to requirements. Each use case of EstateFlow was tested including login, property browsing, co-investment, analytics, and 3D visualization. The tests confirmed that system functions behave correctly for different user roles.

Table 6. 3: Test Case Table

Function	Test Description	Expected Result	Result
Login	User logs in	Dashboard displayed	Pass
Register	New user registration	Account created	Pass
View Properties	Load property list	Properties displayed	Pass
Co-Invest	Enter investment amount	Investment created	Pass
Upload Property	Seller uploads property	Property saved	Pass

Upload 3D Model	Freelancer uploads model	Model linked	Pass
Analytics	View analytics	ROI displayed	Pass
Profit/Loss	Calculate returns	Profit shown	Pass

Table 6.3 presents the overall test cases executed to validate the major functional operations of the EstateFlow system. Each function is tested with a specific scenario to ensure it produces the expected result. The table highlights whether the system behaves correctly under normal usage conditions. It serves as a final verification step to confirm that all integrated features are working smoothly and meet the system requirements.

6.1.4 Integration Testing

Integration testing was performed to ensure communication between Flutter frontend and Firebase backend services. Modules such as authentication, Firestore database, and cloud storage were tested together. The testing verified that data entered in UI is stored in Firebase and retrieved correctly.

Table 6. 4: Integration Testing Table

Module Integration	Test Description	Expected Result	Result
Flutter + Firebase Auth	Login request	User authenticated	Pass
Flutter + Firestore	Upload property	Data stored	Pass
Flutter + Storage	Upload model	File stored	Pass
Investment + Firestore	Save investment	Record created	Pass
Analytics + Database	Fetch data	Analytics generated	Pass
3D Viewer + Storage	Load model	Model displayed	Pass

Table 6.4 presents the integration testing performed to verify the interaction between different system modules and external services. Each test ensures that combined components work seamlessly together and exchange data correctly. The expected results confirm proper communication and functionality across integrated modules. This table helps validate that the system operates reliably as a unified whole rather than as isolated components.

6.2 Tools and Technologies

Testing of EstateFlow mobile application was conducted using various tools and technologies. Flutter framework was used to test UI behavior. Firebase console was used to monitor authentication, database, and storage. Android Emulator and physical devices were used to test application performance. Debugging tools in Android Studio and Visual Studio Code helped identify errors. Testing tools like Postman and GitHub enhanced quality assurance, version control, and collaboration throughout the process:

Table 6. 5: Tools and Technologies Table

Tool	Purpose
Flutter	Mobile UI testing
Firebase Console	Backend testing
Android Studio	Emulator testing
Visual Studio Code	Development & debugging
Firebase Authentication	Login testing
Cloud Firestore	Database testing
Firebase Storage	File upload testing
Android Device	Real device testing

Table 6.5 lists the tools and technologies used during the testing and development of the EstateFlow system. Each tool serves a specific purpose, ranging from user interface testing to backend validation and real-device execution. The table highlights how different technologies contribute to ensuring system functionality, performance, and reliability. It provides an overview of the testing environment and the resources utilized throughout the development process.

6.3 Summary

Testing and evaluation of the EstateFlow mobile application confirmed that all system modules function correctly. Manual testing verified user workflows such as login, property upload, co-investment, and 3D visualization. System testing validated the entire application, while unit testing ensured correctness of individual functions. Functional testing confirmed that each feature meets system requirements. Integration testing verified communication between Flutter frontend and Firebase backend services.

The results show that EstateFlow is stable, reliable, and performs efficiently. The application successfully handles user authentication, property management, co-investment, analytics, ledger generation, and 3D visualization. Testing also confirmed proper data synchronization with Firebase and smooth user interaction. Therefore, the EstateFlow mobile application meets functional and performance requirements and is ready for deployment.

Chapter 7

CONCLUSION AND FUTURE WORK

7 Conclusion

The EstateFlow mobile application was developed to address the limitations of traditional real estate platforms by introducing a collaborative and interactive property investment system. The project focused on enabling users to explore properties in 3D, participate in co-investment opportunities, track financial performance, and manage real estate portfolios using a single mobile application. By integrating Flutter framework with Firebase backend services, the system provides real-time data synchronization, secure authentication, and scalable architecture suitable for modern mobile environments.

Throughout the development process, the system was designed using structured software engineering practices including requirement analysis, UML modeling, architecture design, and modular implementation. The application supports multiple user roles including Buyer, Investor, Seller/Broker, and 3D Freelancer. Each role interacts with the system through role-based dashboards and functionalities. Buyers and investors can browse property listings, explore properties in 3D view, and invest collaboratively. Sellers and brokers can upload and manage property listings while generating financial ledger reports. Freelancers can upload 3D models to enhance property visualization.

The integration of Firebase Authentication ensures secure login and role-based access control. Cloud Firestore database enables structured storage of user information, property details, and investment records. Firebase Cloud Storage provides secure storage for property images and 3D models. The Flutter-based mobile interface ensures smooth navigation, responsive layout, and user-friendly interaction. These components work together to create a unified real estate ecosystem.

The system also introduces co-investment functionality, which allows multiple users to invest in a single property. This feature reduces financial barriers and enables shared ownership. Investment analytics and profit/loss tracking modules provide transparency and help users evaluate their investment performance. The ledger generation feature maintains transaction records and supports financial accountability. Additionally, the 3D visualization module improves user decision-making by providing immersive property exploration.

Testing and evaluation were performed to validate the functionality and performance of the application. Manual testing confirmed that all modules operate correctly. System testing verified complete workflows, while unit testing validated individual functions. Functional

testing ensured that all user requirements were met. Integration testing confirmed smooth communication between Flutter frontend and Firebase backend services. The results demonstrated that the EstateFlow application is reliable, efficient, and scalable.

Overall, the EstateFlow project successfully achieves its objective of providing a mobile-based real estate co-investment platform with 3D visualization and financial tracking. The system improves accessibility, transparency, and user experience in real estate investment. By combining mobile technology, cloud services, and interactive visualization, EstateFlow presents a modern solution for collaborative property investment and management.

7.1 Future Work

The EstateFlow mobile application provides a strong foundation for collaborative real estate investment and 3D property visualization. However, the system can be further enhanced by integrating advanced features and improving scalability. The following future enhancements can improve functionality, performance, and user experience.

1. Online Payment Integration

Future versions of EstateFlow can include secure online payment integration to allow users to invest directly through the mobile application. Payment gateways such as credit/debit card processing, mobile banking, and digital wallets can be incorporated. This feature would automate investment transactions and reduce manual entry of investment amounts. The system could also generate transaction receipts and maintain payment history. Secure payment processing with encryption and authentication would improve trust and reliability. This enhancement would transform EstateFlow into a complete real estate investment platform.

2. AI-Based Property Recommendation System

An AI-powered recommendation system can be added to analyze user preferences and investment behavior. Machine learning algorithms can evaluate user activity, location preferences, budget range, and investment history. Based on this data, the system can recommend suitable properties. This feature would improve decision-making and reduce search time for users. Predictive analytics can also forecast property value trends and potential returns. This enhancement would make EstateFlow more intelligent and personalized.

3. Real-Time Chat and Communication Module

A real-time messaging system can be implemented to allow communication between buyers, investors, and sellers. This feature would enable users to discuss property details, negotiate investment terms, and ask questions directly within the application. The chat module can support notifications, message history, and file sharing. Real-time communication would improve collaboration and user engagement. It would also reduce dependency on external communication platforms.

4. Advanced Analytics and Reporting Dashboard

Future development can include advanced analytics dashboards with graphical visualization. Charts and graphs can display investment growth, ROI trends, and portfolio distribution. Comparative analytics can help users evaluate multiple properties. Sellers can analyze investor participation and property performance. These insights would help users make better investment decisions. The reporting module could also export data in PDF or Excel format.

5. Augmented Reality (AR) and Enhanced 3D Visualization

The 3D visualization feature can be enhanced using augmented reality technology. Users could view properties in real-world environments using mobile camera. AR-based walkthroughs would provide immersive experience. Additionally, advanced 3D controls such as floor navigation, room labeling, and measurement tools can be added. This feature would improve property understanding and increase user confidence before investing.

Chapter 8

REFERENCE

8 References

1. Google. (2024). Flutter Documentation. Retrieved from <https://docs.flutter.dev>
→ Reference for building cross-platform mobile UI using Flutter framework and Dart programming language.
2. Google. (2024). Dart Programming Language Documentation. Retrieved from <https://dart.dev/guides>
→ Used for implementing application logic and object-oriented programming in EstateFlow mobile application.
3. Google Firebase. (2024). Firebase Authentication Documentation. Retrieved from <https://firebase.google.com/docs/auth>
→ Reference for implementing secure user authentication and role-based login system.
4. Google Firebase. (2024). Cloud Firestore Documentation. Retrieved from <https://firebase.google.com/docs/firestore>
→ Used for storing users, properties, investments, analytics, and ledger data.
5. Google Firebase. (2024). Firebase Cloud Storage Documentation. Retrieved from <https://firebase.google.com/docs/storage>
→ Reference for uploading and managing property images and 3D models.
6. Flutter Community. (2024). Flutter 3D Viewer Libraries Documentation. Retrieved from <https://pub.dev>
→ Used for implementing interactive 3D property visualization within mobile application.
7. Android Developers. (2024). Android Application Development Guidelines. Retrieved from <https://developer.android.com/docs>
→ Reference for Android app structure, permissions, and deployment requirements.
8. Visual Studio Code. (2024). Visual Studio Code Documentation. Retrieved from <https://code.visualstudio.com/docs>
→ Used as development environment for writing Flutter and Dart code.
9. GitHub. (2024). GitHub Documentation. Retrieved from <https://docs.github.com>
→ Used for version control, project repository management, and collaboration.